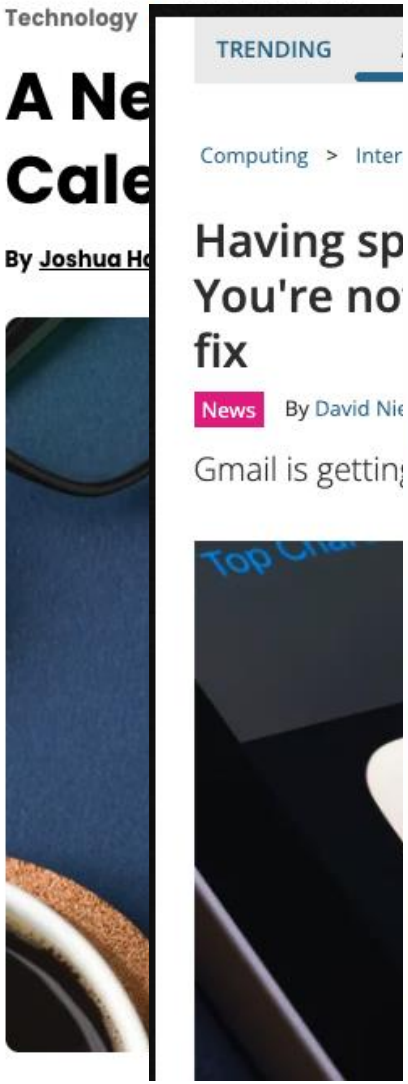
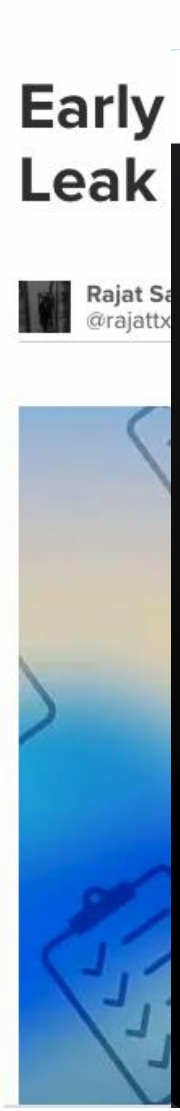
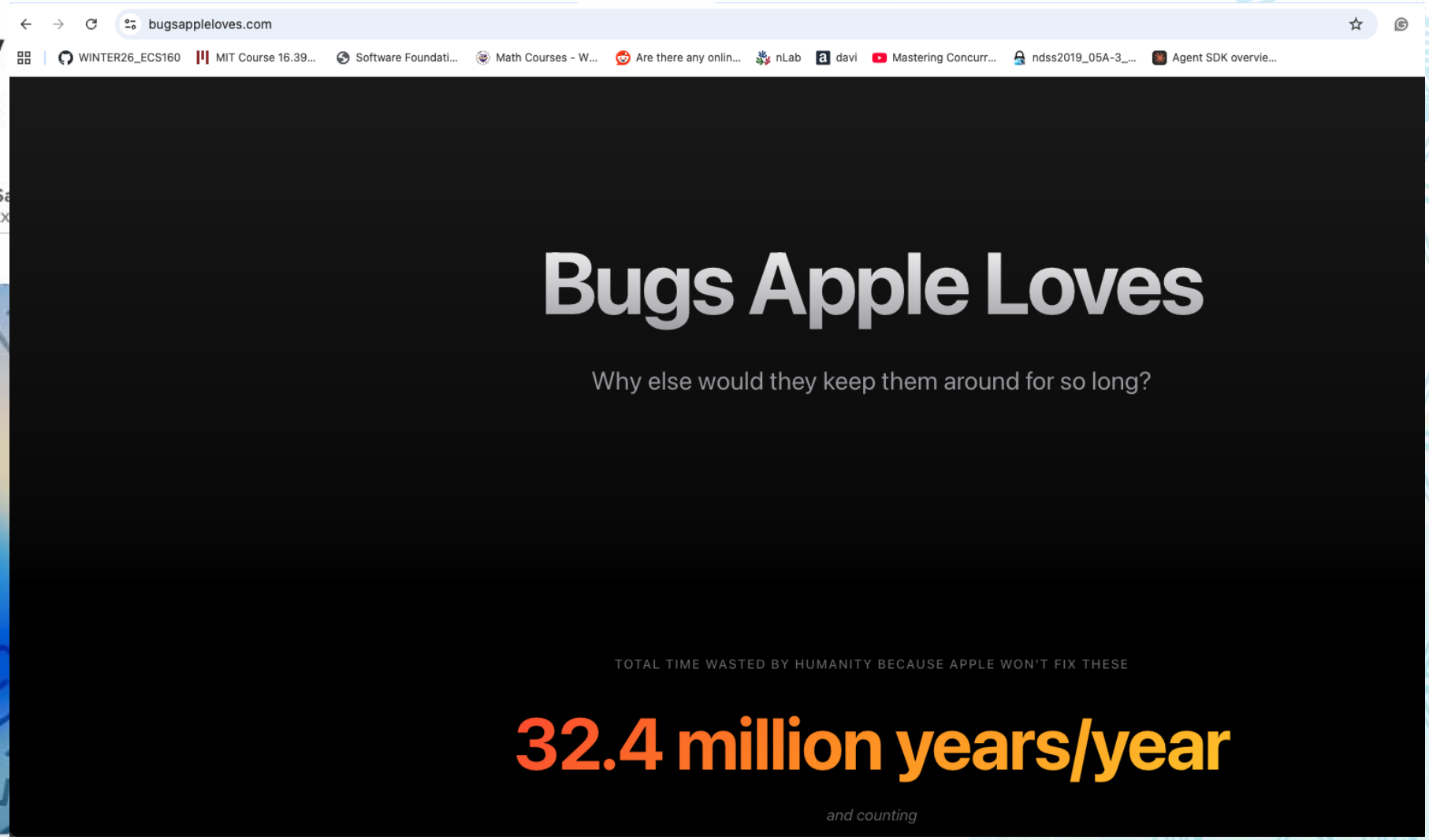
Software Testing

Tapti Palit

Software has bugs







Some bugs are security vulnerabilities

cybernews.com/security/high-severity

26_ECS160 MIT Course 16.39... Soft

Publi

Image

NGII
enab
redir

Malware

NEWS

Upda
day b
execu
webp

by Pieter Ar

Google has
in how the

CVE-2026-
it serious e
release.

Home > News > Microsoft > Microsoft patches actively exploited Office zero-day vulnerability

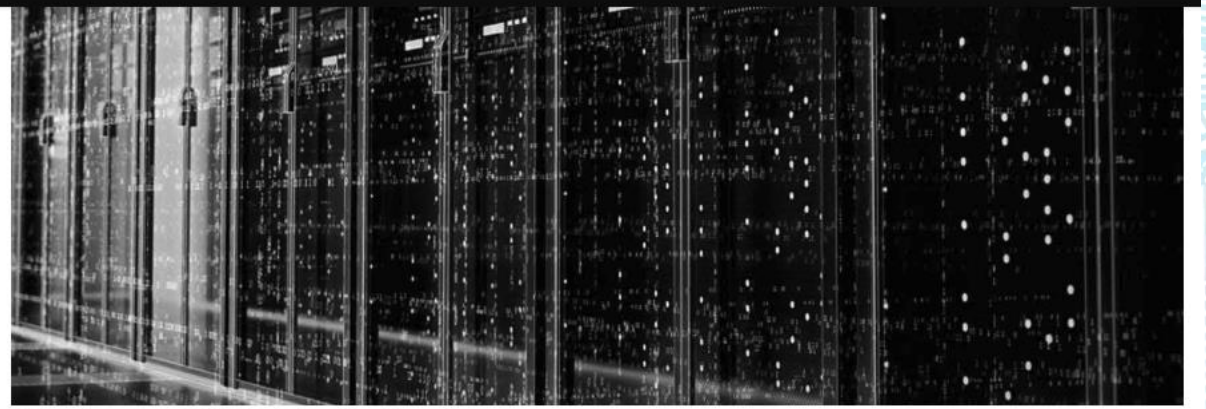
Micro

By Sergiu

Microsoft has released emergency out-of-band security updates to patch a high-severity Microsoft Office

CYBERCRIME MAGAZINE

ABOUT RESEARCH NEWS



Cybercrime Costs. PHOTO: Cybercrime Magazine.

Cybercrime To Cost The World $10.5 Trillion Annually By 2025

Special Report: Cyberwarfare In The C-Suite.

– *Steve Morgan, Editor-in-Chief*

Sausalito, Calif. – Nov. 13, 2020

If it were measured as a country, then cybercrime — which is predicted to inflict damages totaling $6 trillion USD globally in 2021 — would be the world's third-largest

Software testing

- Process of finding bugs and vulnerabilities in software
- *"Program testing can be used to show the presence of bugs, but never to show their absence"*
  - By E. W. Dijkstra

Types of software testing

- Testing based on dynamic analysis
  - Software is executed
  - Monitor for assertion failures and other bugs during execution
  - Unit testing (JUnit), integration testing, ***fuzz testing***
- Testing based on static analysis
  - Source code is analyzed without execution
  - Analysis identifies known bug patterns (uninitialized variables, unsafe data flow, null pointer dereferences)
  - ***Dataflow analysis, control flow analysis,*** abstract interpretation
- Hybrid approach: ***symbolic execution***

Dynamic Testing

Recap: unit testing with JUnit

- Test individual functions
- Add assertions to check if post-conditions hold
- Test corner-cases
- Manually created

```
public Car {
    // --snip
    public void accelerate() {
        this.speed += 10;
    }
}

public class CarTest {
    @Test
    public void testAccelerate() {
        Car car = new Car("Toyota", 50);
        car.accelerate();
        assertEquals(car.getSpeed(), 60);
    }
}
```

Hard to cover all code with unit testing

- Linux kernel
  - ~30 million lines of code
- Google's chromium browser
  - ~28 million lines of code
- Hard to manually write unit tests to cover all this code
  - ***Need automated testing***
- Automated testing is a sub-field of automated software engineering

Silent failures

- Some bugs lead to crashes, assertion failures
  - Detecting crash bugs requires exercising the buggy code with the right inputs
  - E.g, `assert(i > 0)`
- Some bugs are silent

Silent failures and UBs

C/C++

```
#include <stdio.h>

int main(int argc, char** argv) {
    char a = ‘A’;
    char arr[4] = {‘a’, ‘b’, ‘c’,
‘d’};
    int index = atoi(argv[1]);
    printf(“%c\n”, arr[index]);

}

> gcc test.c –o test
> ./test 10
> Output?
```

Java

```
class MyApp {
    public static void main(String[] args) {
        char a = ‘A’;
        char arr[] = {‘a’, ‘b’, ‘c’, ‘d’};
        int index = Integer.parseInt(args[0]);
        System.out.println(arr[index]);
    }
}

> javac MyApp.java
> java MyApp.class 10
> # Output?
```

Silent failures and UBs

C/C++

```
#include <stdio.h>

int main(int argc, char** argv) {
    char a = ‘A’;
    char arr[4] = {‘a’, ‘b’, ‘c’,
‘d’};
    int index = atoi(argv[1]);
    printf(“%c\n”, arr[index]);

}

> gcc test.c -o test
> ./test 10
> ?
```

Java

```
class MyApp {
    public static void main(String[] args) {
        char a = ‘A’;
        char arr[] = {‘a’, ‘b’, ‘c’, ‘d’};
        int index = Integer.parseInt(args[0]);
        System.out.println(arr[index]);
    }
}

> javac MyApp.java
> java MyApp.class 10
> Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 10
out of bounds for length 4
```

Undefined Behavior (UB)

Memory safety

- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow

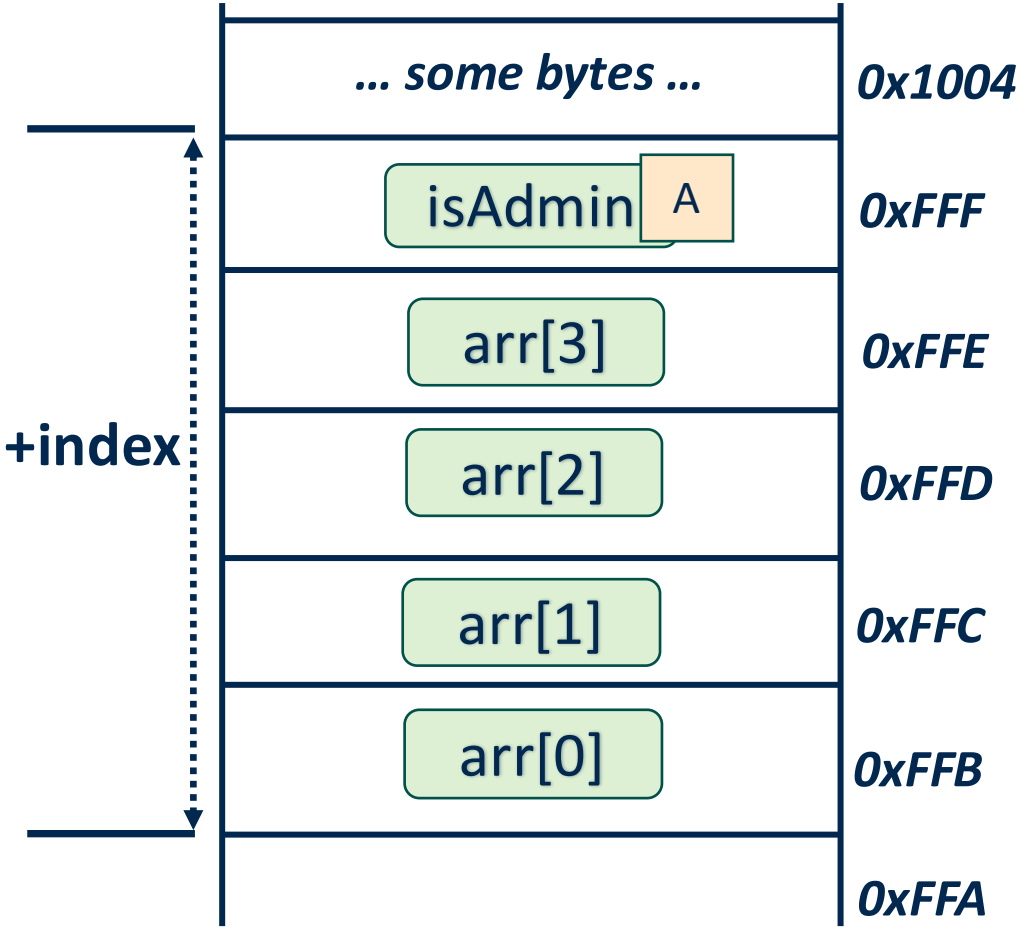
```
int main(int argc, char** argv) {
    int isAdmin = 0;
    char arr[4];

    int index = atoi(argv[1]);

    arr[index] = 'A';
    // --snip
}
```

Can silently corrupt the isAdmin variable!

Memory safety




```
int main(int argc, char** argv) {
    char isAdmin = check_admin_access();
    char arr[4];

    int index = atoi(argv[1]);

    arr[index] = 'A';

    if (isAdmin) {
        delete_all_files();
    }
}
```

Memory safety

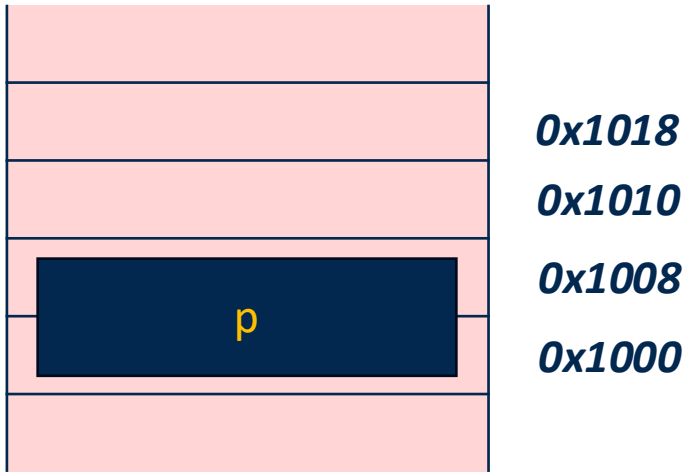
- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow
- Use-after-free

```
int main(int argc, char** argv) {
    char* p = malloc(100);
    // do stuff with p
    free(p);
    // 1000 program statements
    char* passwd = malloc(20);

    p[5] = 'X';
}
```

UAF bug!

UAF bug details

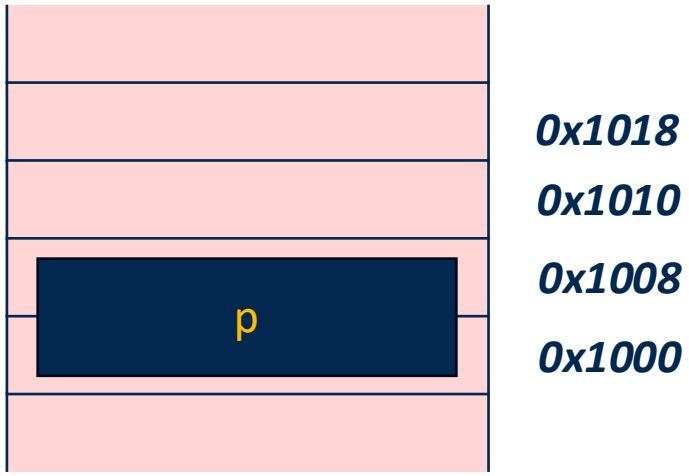


Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

UAF bug details

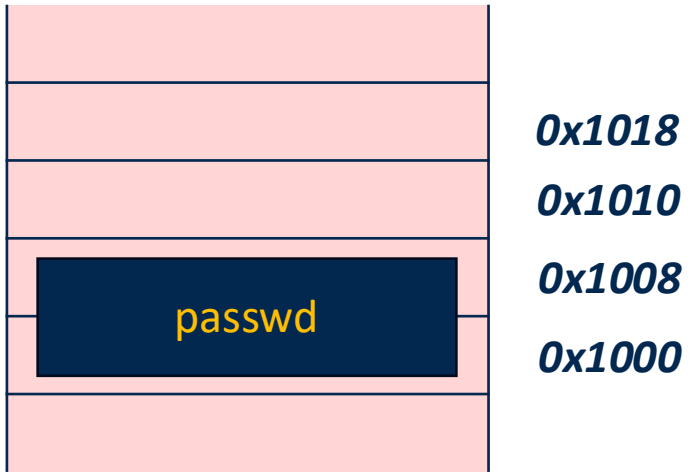


Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

UAF bug details



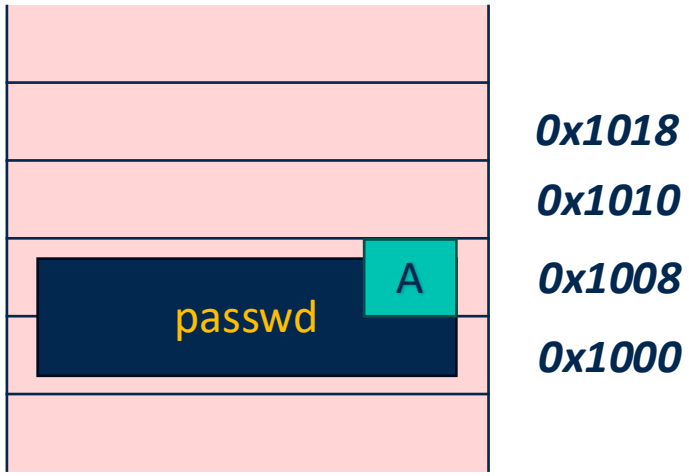
Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

Attacker manipulates the heap to make sensitive heap objects be allocated at the site with UAF pointer accesses

UAF bug details



Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);

    p[i] = 'X';
}
```

UAF write corrupts the sensitive data object

Memory safety

- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow
- Use-after-free
- Double-free
- None of these bugs necessarily crash the program!

```
int main(int argc, char** argv) {
    char* p = malloc(100);
    // do stuff with p
    free(p);

    char* passwd = malloc(20);

    free(p);
}
```

Crash and silent failures

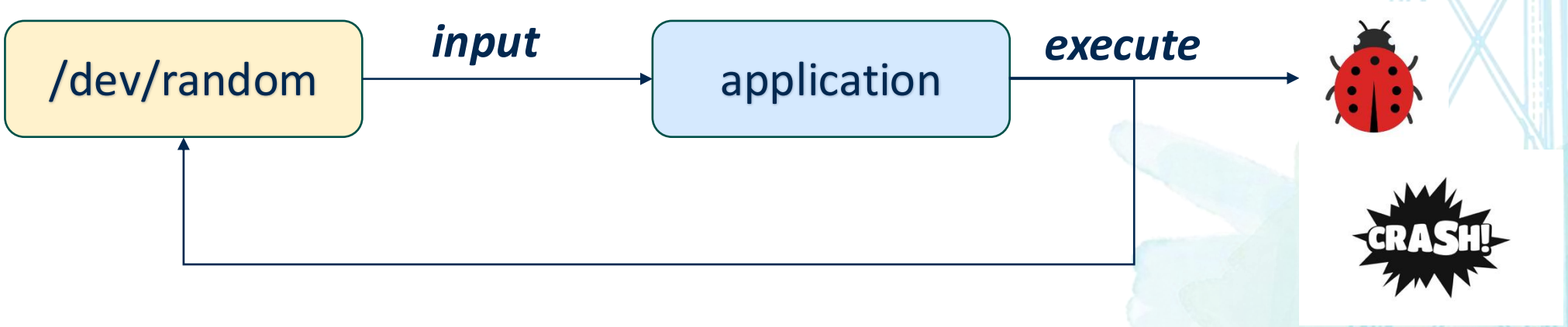
- Some bugs lead to crashes, assertion failures
  - Detecting crash bugs requires exercising the buggy code with the right inputs
  - E.g, `assert(i > 0)`
- Some bugs are silent
  - Detecting silent (UB) bugs requires both, exercising the buggy code with the right inputs ***AND an oracle to identify the buggy execution***

Fuzz testing

- Goal: to find application inputs that reveal bugs
- Test the entire application instead of individual functions
- Generate inputs randomly until an error is uncovered
  - ***How to generate new inputs?***

Fuzz testing

Source for random input on *nix systems



Black-box testing

Types of bugs

- Incorrect argument validation, incorrect type casting
- Buffer overflows, memory leak, div-by-zero, use-after-free, segmentation fault, and so on…
- Logic errors through assertion failures
- Applications written in any language can be fuzzed

Blackbox fuzzing design

- Simple loop invoking app with new inputs
- No internal visibility into the app

```
int main(int argc, char* argv[]) {
  int sum = process_file(argv[1]);
  printf("%d\n", sum);
}
```

```
int process_file(const char *p){
  FILE *f=fopen(p,"r"); if(!f) return -1;
  char s[256]; int sum=0;
  while(fgets(s,sizeof s,f)){
    if(s[0]=='#'||s[0]=='\n') continue;
    else if(s[0]=='A') sum+=1;
    else if(s[0]=='B') sum*=2;
    else if(s[0]=='C' && s[1]=='!') { fclose(f);
return -2; }
  }
  fclose(f); return sum;
}
```

Fuzzing target

Blackbox fuzzing design

- Simple loop invoking app with new inputs
- No internal visibility into the app
- E.g., simple blackbox fuzzing loop

```
int main(int argc, char* argv[]) {
  char* random_bytes = read("/dev/random");
  write("/tmp/f.dat", random_bytes);
  process_file("/tmp/f.dat");
}

int process_file(const char *p){
  FILE *f=fopen(p,"r"); if(!f) return -1;
  char s[256]; int sum=0;
  while(fgets(s,sizeof s,f)){
    if(s[0]=='#'||s[0]=='\n') continue;
    else if(s[0]=='A') sum+=1;
    else if(s[0]=='B') sum*=2;
    else if(s[0]=='C' && s[1]=='!') { fclose(f);
return -2; }
  }
  fclose(f); return sum;
}
```

Fuzzing target

Pros and cons

- Pros: generating random inputs is simple
- Cons: random inputs are not good inputs
- E.g., fuzzing a SQL library with SQL commands
  - Real inputs: `CREATE TABLE mytable AS (id INTEGER, … );`
  - Random inputs: `aVkjW3txpLZ044zo5kLdUlsU2MtlLNkhwxI8Aew7c0KPfTS115i2rzfBlgod`
  - The database will reject such statements as invalid SQL and not execute the SQL command -> the SQL code will not even be tested

Pros and cons

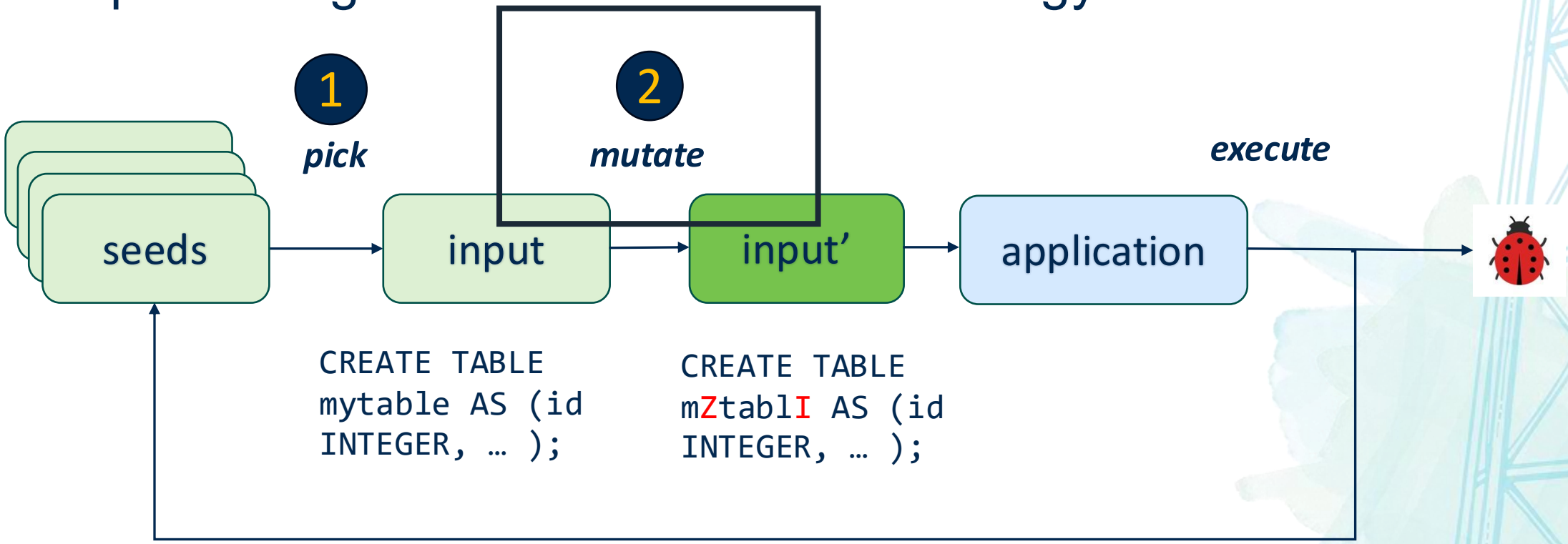
- Real inputs: `CREATE TABLE mytable AS (id INTEGER, … );`
- Random inputs: `aVkjW3txpLZ044zo5kLdUlsU2MtlLNkhwxI8Aew7c0KPfTS115i2rzfBlgod`
- The database will reject such statements as invalid SQL and not execute the SQL command -> the SQL code will not even be tested

```
int process_sql(const char *query) {
    if (!check_if_sql_valid(query))
        return ERROR_INVALID;

    // Deep logic never reached by
    // most mutated inputs
    execute_query(query);
}
```

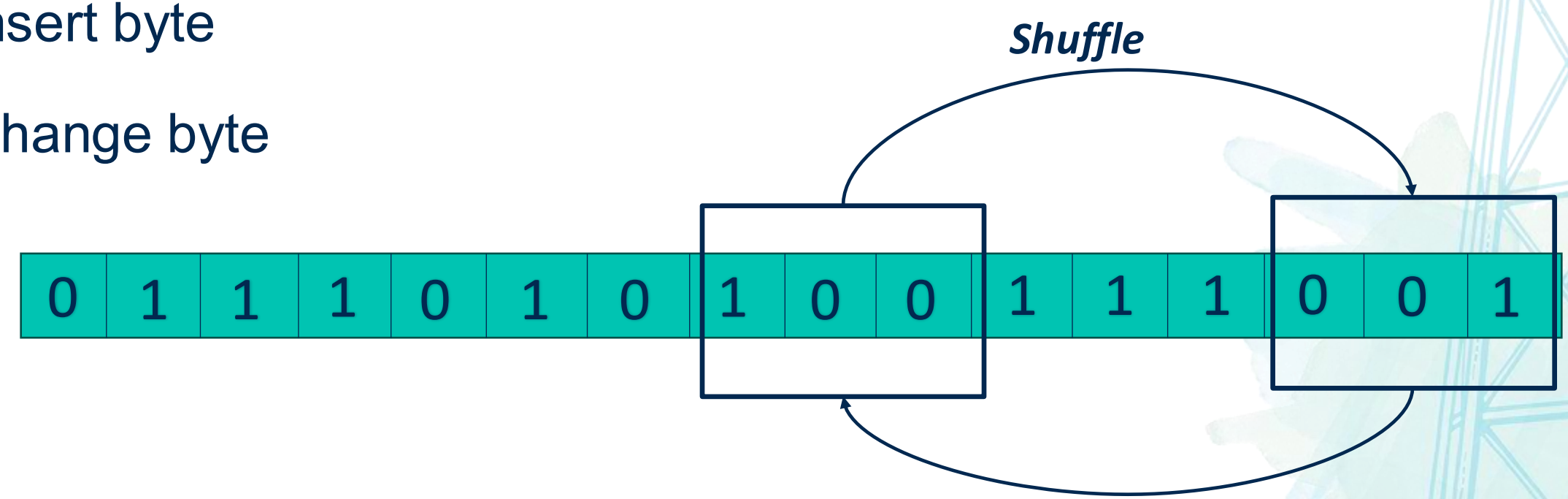
Mutation fuzzer - generate inputs via mutation

- Start with some known valid inputs called *"seeds"*
- Keep mutating them based on some "strategy"



Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte



Mutation strategy – byte-level transformations

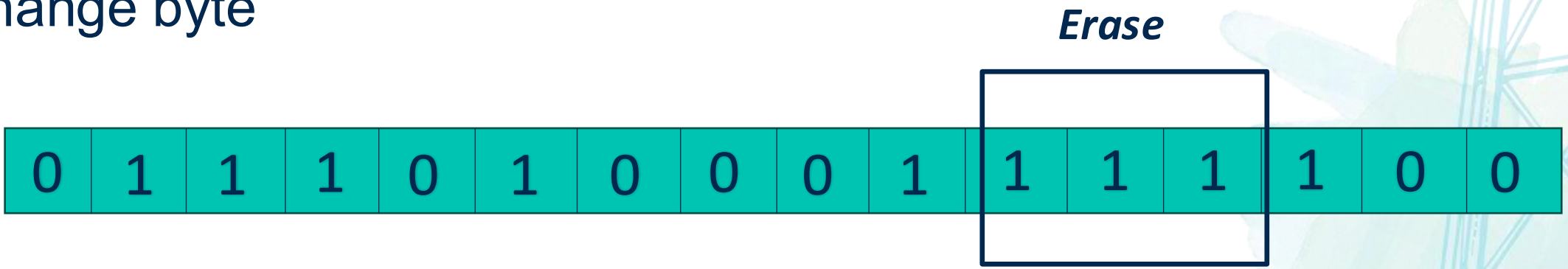
- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte

New input

0	1	1	1	0	1	0	0	0	1	1	1	1	1	0	0

Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte



Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte

Next input

0	1	1	1	0	1	0	0	0	1	1	0	0

Mutation strategy – byte-level transformations

- Benefits:
  - Simple to implement
  - Good for binary inputs and streams
    - JPEG file encoder/decoder
- Disadvantages:
  - Generates many invalid inputs for applications that expect structured inputs

Mutation strategy – dictionaries

- Programmer provides a dictionary of “keywords”
  - For SQL, could include `CREATE`, `DELETE`, `INSERT`, `INTO`, `FROM`, `INTEGER`, and so on…
  - For HTML, could include HTML tags `<html>`, `<body>`, `<p>`, and so on…
- Mutate the input by replacing or adding a chosen keyword from this dictionary
- Input - `<html><body><p>Hello world </p></body></html>`
  Dictionary - `{<html>, <body>, <p>,…}`
  Mutated input –
  `<html><body><p> Hello world <p>, Welcome!</p> </p></html>`

Mutation strategy – dictionaries

- Benefits:
  - Still simple to implement
  - Works reasonably well for structured inputs
- Disadvantages
  - Still generates many invalid inputs for applications that expect structured inputs

Grammar-aware fuzzing

- Previous mutators generate many syntactically invalid inputs
- For structured formats (SQL, HTML, JSON), the parser rejects them before reaching deeper application logic
- Wasted fuzzing effort
  - Need a way to guarantee valid input structure

```
int process_sql(const char *query) {
    if (!check_if_sql_valid(query))
        return ERROR_INVALID;

    // Deep logic never reached by
    // most mutated inputs
    execute_query(query);
}
```

Grammar-aware fuzzing

- Define the input format as a context-free grammar (BNF rules)
- Generate inputs by randomly expanding production rules from the start symbol
- Every generated input is syntactically valid *by construction*

Grammar-aware fuzzing

- Define the input format as a context-free grammar (BNF rules)
- Generate inputs by randomly expanding production rules from the start symbol
- Every generated input is syntactically valid *by construction*

Context free grammar for simplified SQL

- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
               WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```

Context free grammar for simplified SQL

- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>     ::= SELECT <columns> FROM <table>
              | SELECT <columns> FROM <table>
                WHERE <cond>
<columns>   ::= <col> | <col>, <columns>
<col>       ::= [A-Za-z]+
<table>     ::= [A-Za-z]+
<cond>      ::= <col> = <value>
<value>     ::= [0-9]+ | '[A-Za-z]+'
```

Context free grammar for simplified SQL

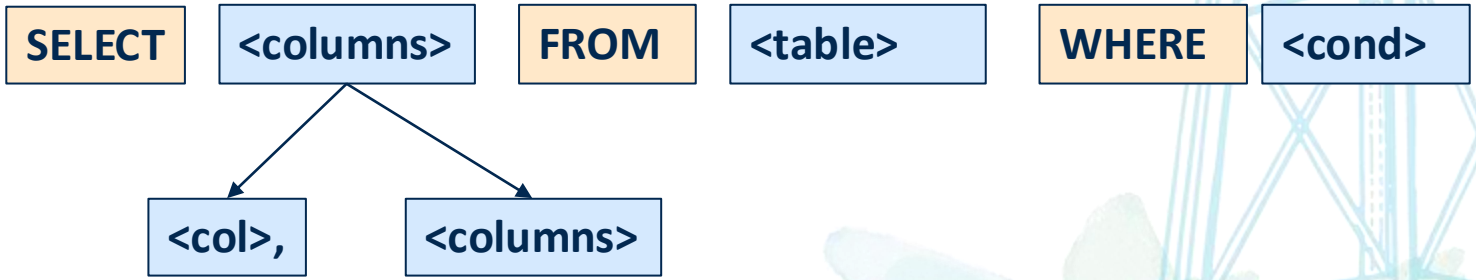
- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```

SELECT | <columns> | FROM | <table> | WHERE | <cond>

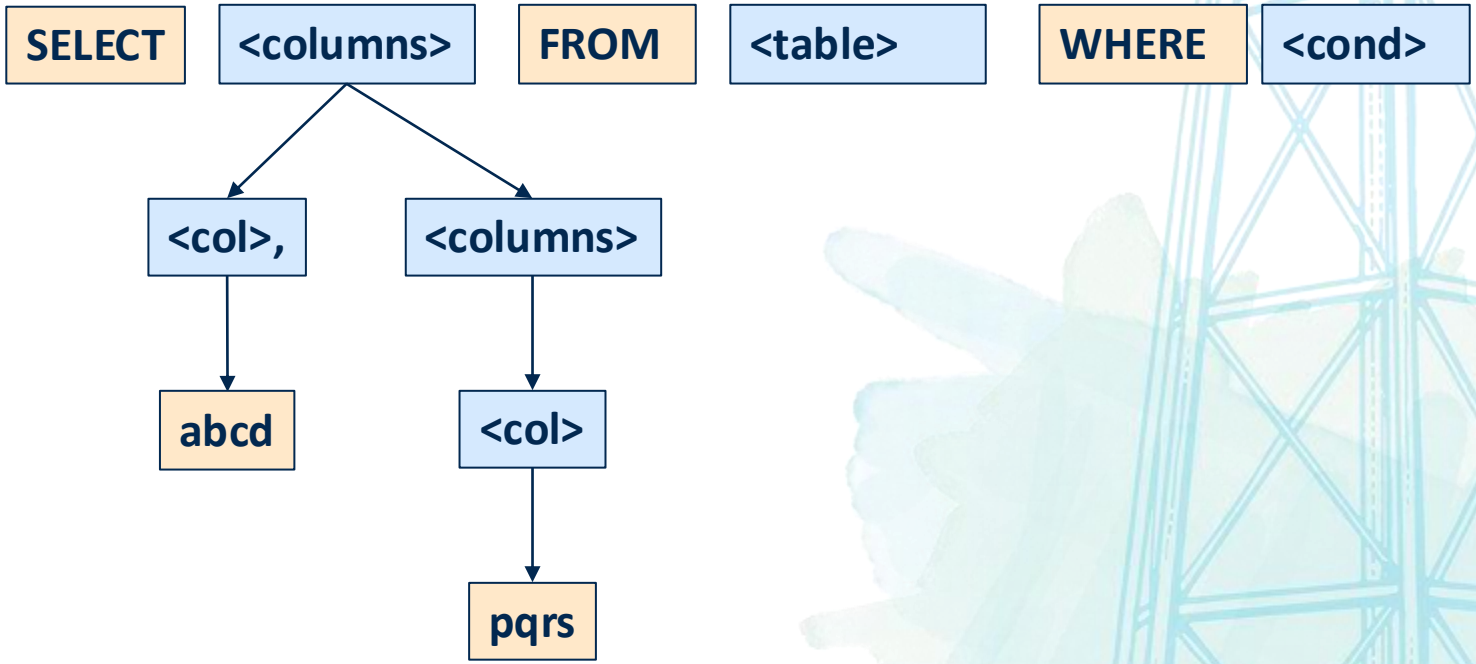
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



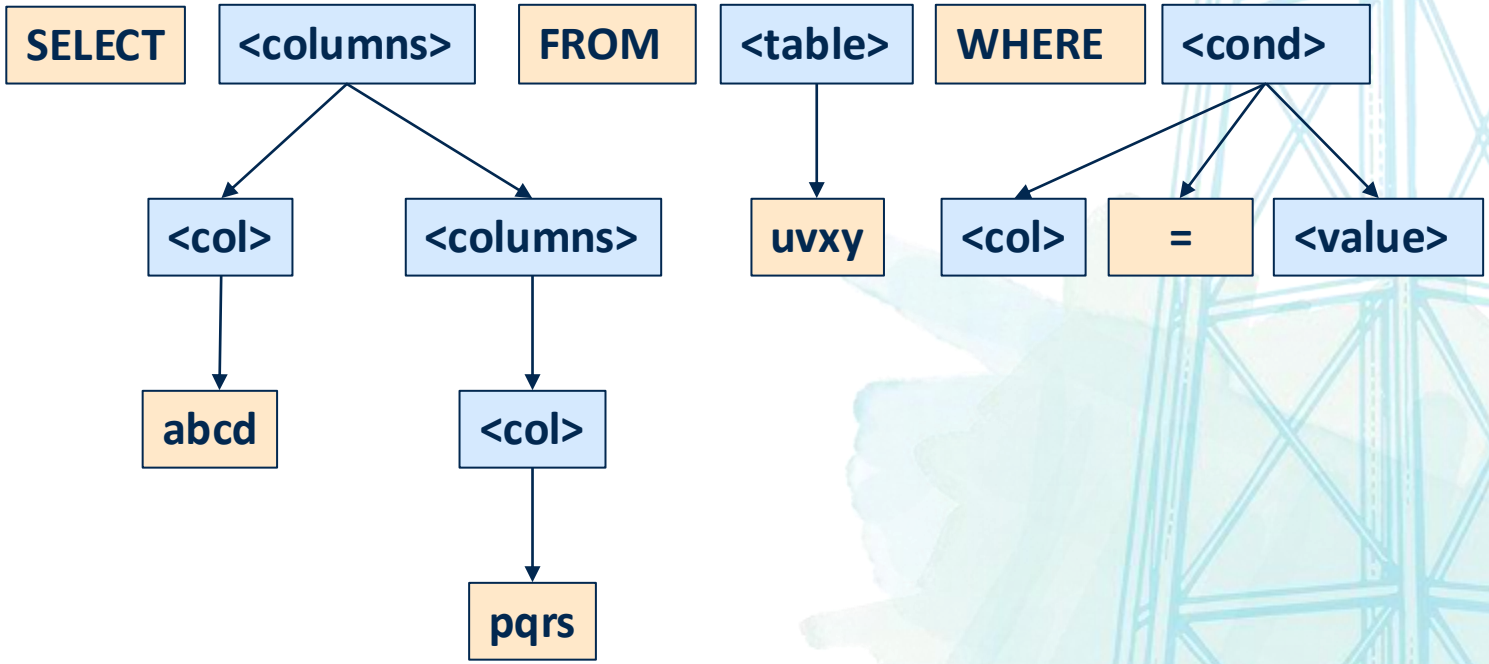
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



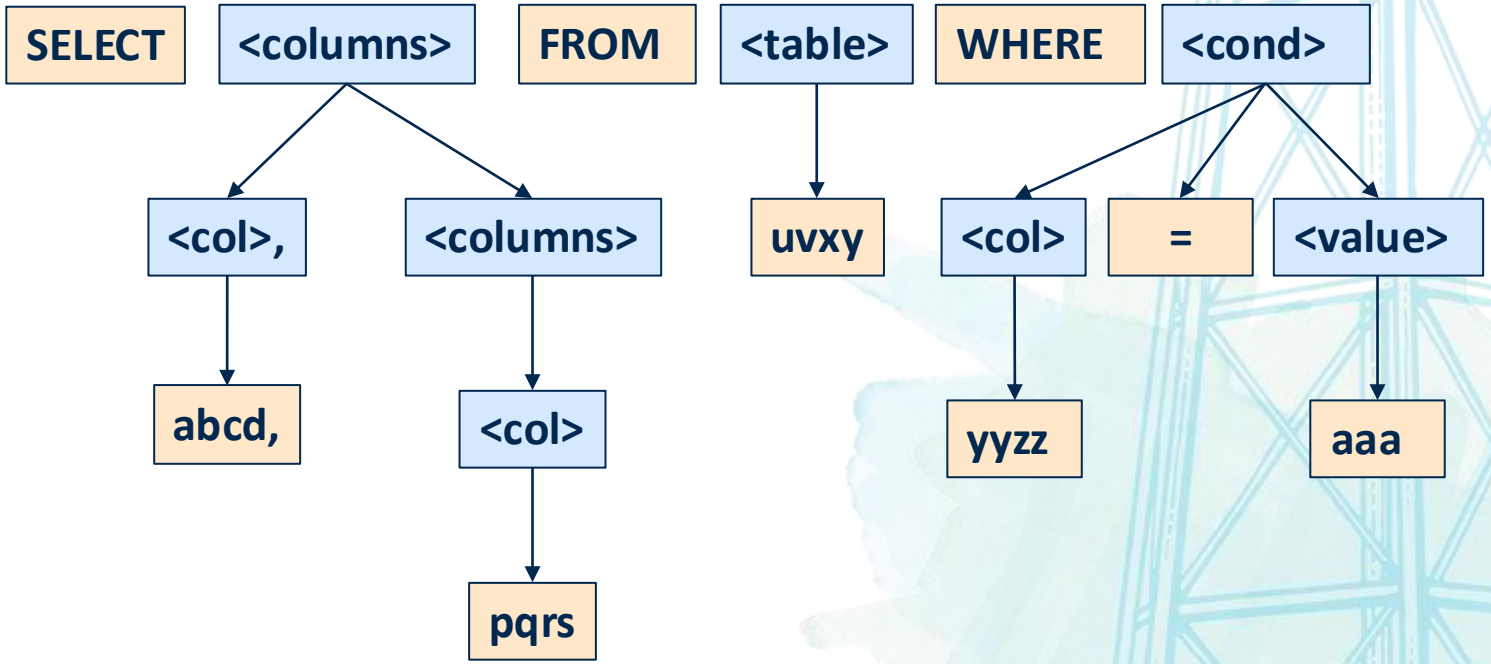
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
               WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



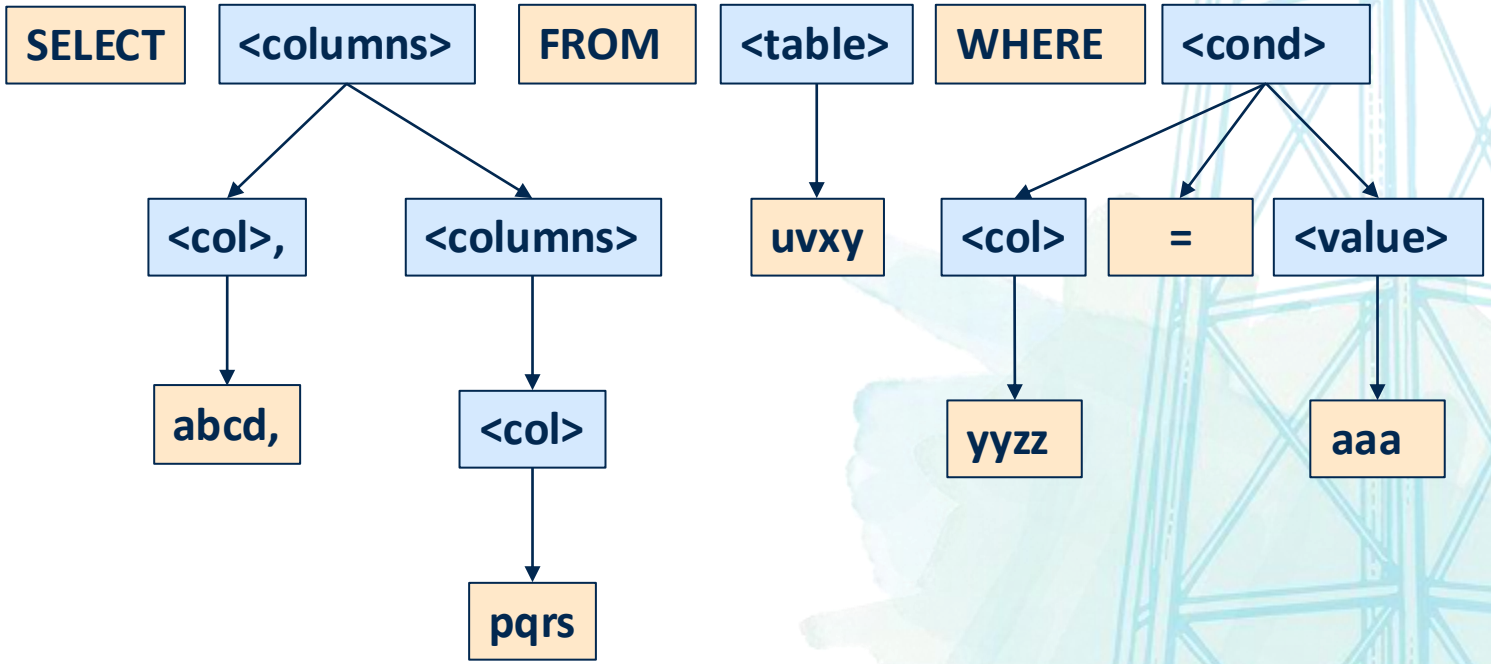
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



Produced SQL statement: SELECT abcd, pqrs FROM uvxy WHERE yyzz = aaa

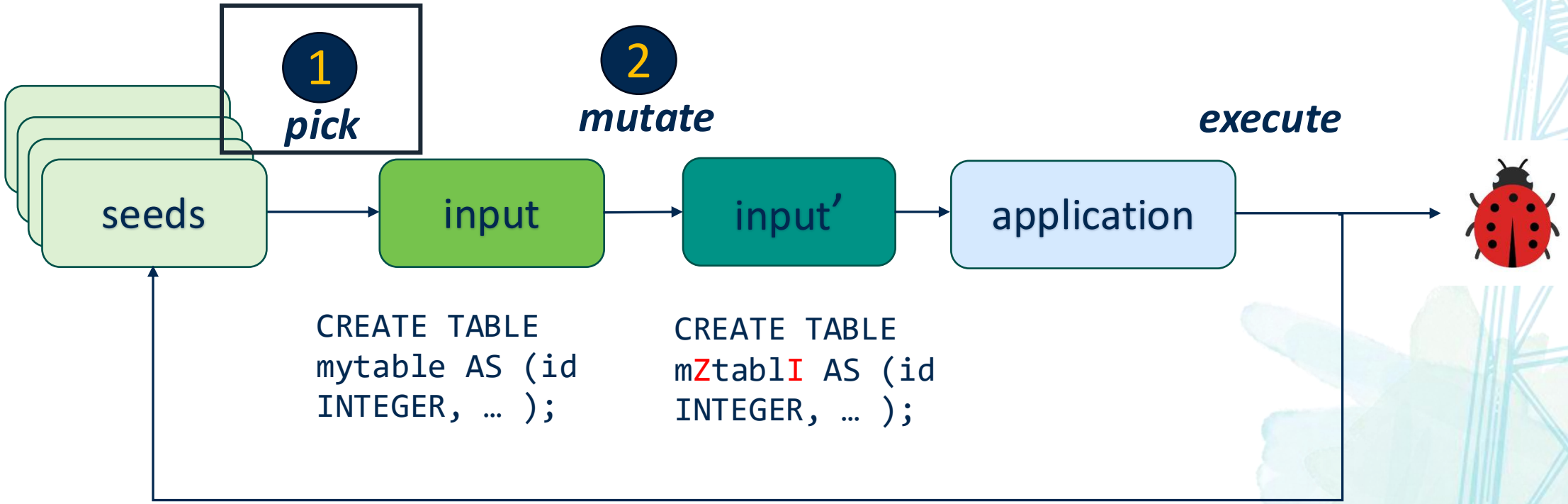
Pros and cons

- Positives:
  - 100% syntactically valid → tests deeper code paths
  - Can weight rules to target specific features
- Negatives
  - Must write/maintain a grammar for each input format
  - Only generates valid inputs — may miss bugs triggered by malformed input
  - Generating and mutating valid inputs is expensive

Fuzzer success factors

- Fuzzer throughput (exec/s)
- Percentage of valid inputs generated by the mutator
- Tradeoff: simpler mutators have higher throughput, but lower number of valid inputs

Fuzzing pipeline



How to pick which seeds to mutate?

- Goal: Should pick seeds that are “most promising” and likely to find bugs
- What is ***“most promising”?***

Metric of seed quality: code coverage

- Code coverage is the simplest metric for measuring seed quality
- Amount of code or execution paths explored during fuzz testing
- Input seeds that increase coverage are “more promising”
  - Explore new program behaviors
  - Ensures untested paths are tested
- High coverage = high bug-finding potential

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “AB”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “AC”
  - No new coverage -> uninteresting seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “BB”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “BA”
  - No new coverage -> uninteresting seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

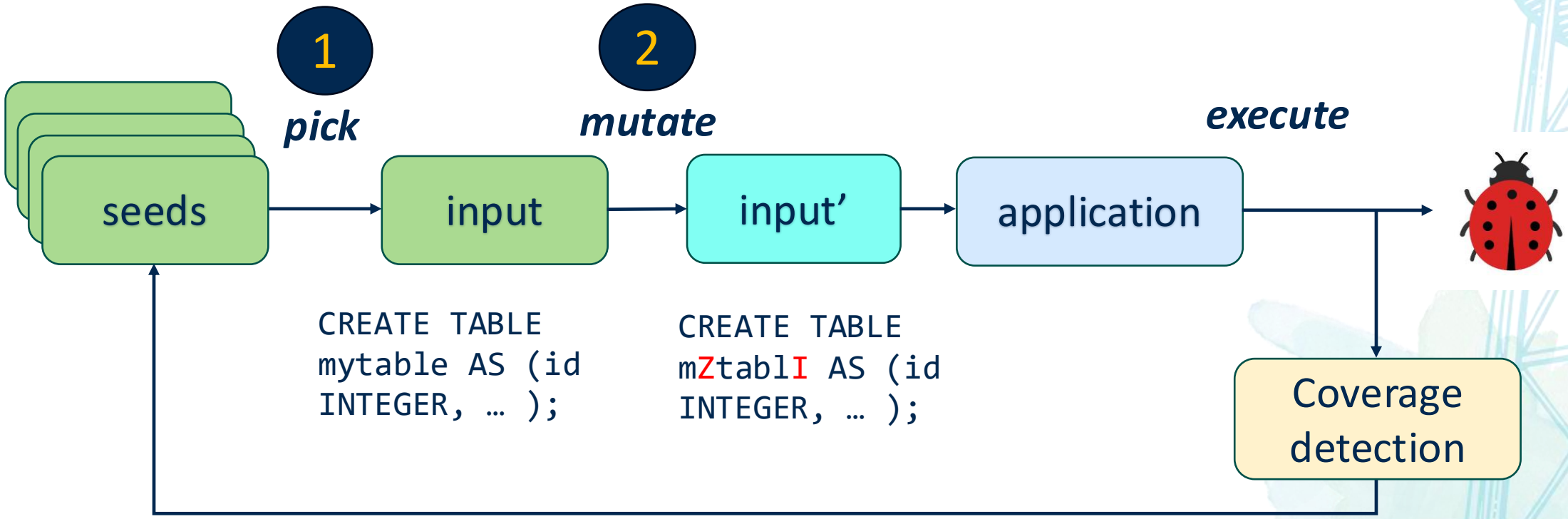
- Amount of code or execution paths explored during fuzz testing
- Input: s = “BC”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Coverage guided fuzzing

- Record the coverage
- Prioritize mutating input seeds which give higher coverage
- Also known as “greybox fuzzing”

Coverage-guided fuzzer pipeline



Add* input' *to* seeds *if new coverage observed

Fuzzing – high level ideas

- Randomly generate inputs
- Mutation fuzzing
  - Start with known valid inputs
  - Mutate them
    - Byte-level transformations
    - Dictionary-based transformations
- Pick inputs which resulted in new branch coverage for further mutation

AFL demo

- American fuzzy lop (AFL)
- Fuzzer for C/C++ applications
- https://github.com/wolframroesler/afl-demo

AFL demo

- Throughput – number of inputs tested per second

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

AFL demo

- Crashes found by the fuzzer

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

Crashes

AFL demo

- Coverage – number of code paths executed

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

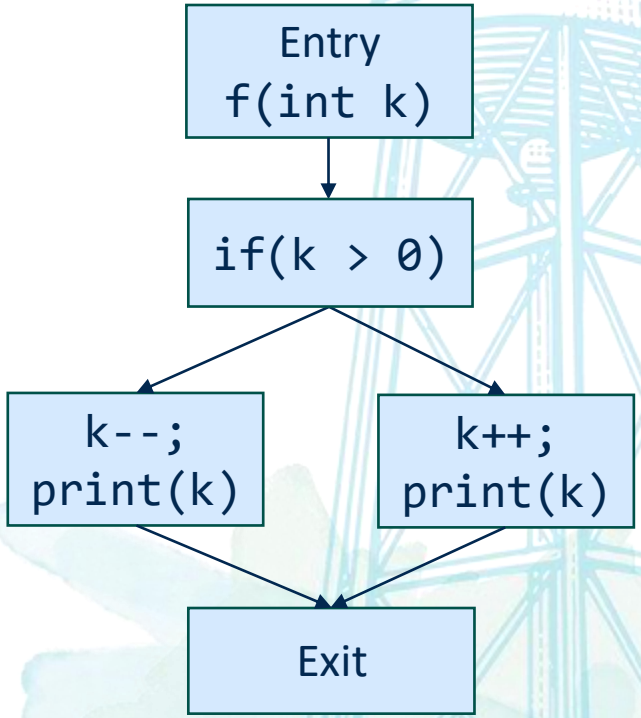
Coverage

Coverage collection

Control flow graph

- A program can have many representations
- Control flow graph is the one of the simplest of representations
- A CFG consists of basic blocks

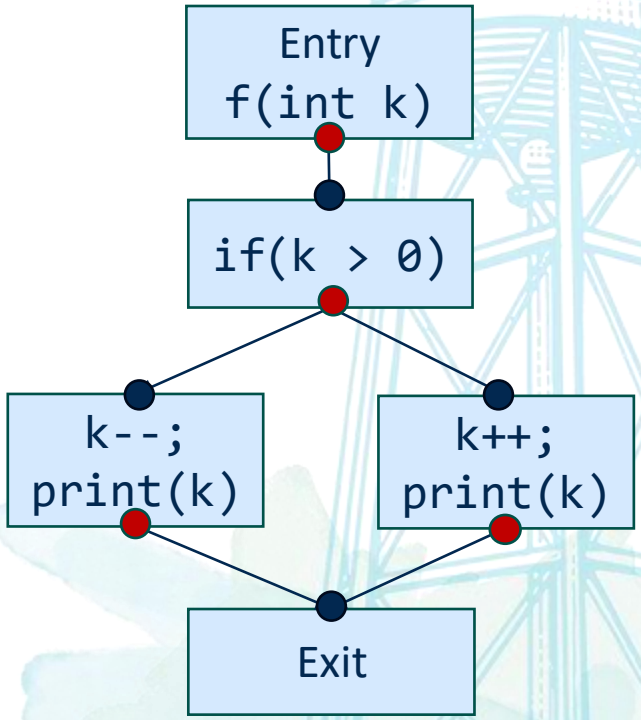
```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```



Basic blocks

- A basic block in program P is a sequence of consecutive statements
- A BB has a single entry and a single exit point

```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```

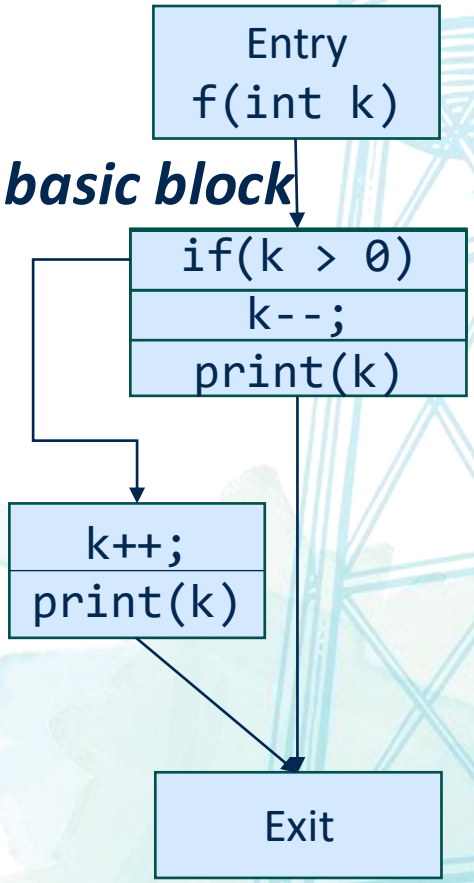


Basic blocks

- A basic block in program P is a sequence of consecutive statements
- A BB has a single entry and a single exit point

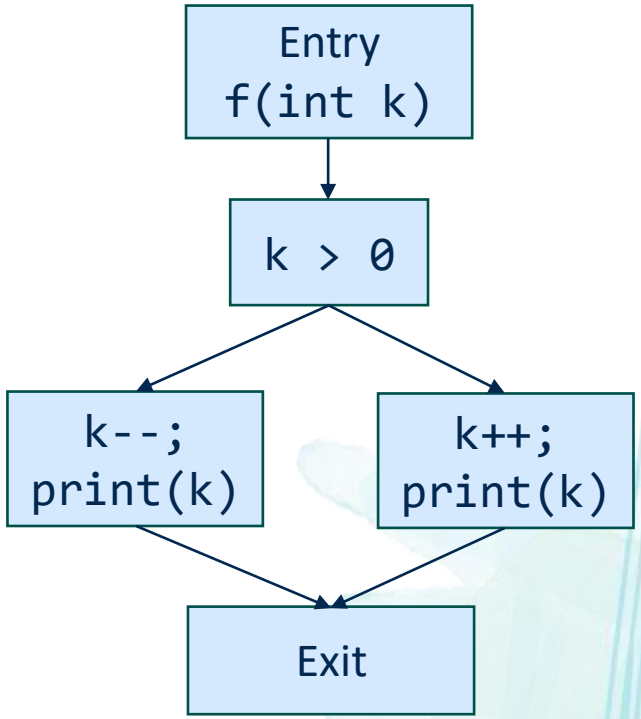
```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```

Not a basic block



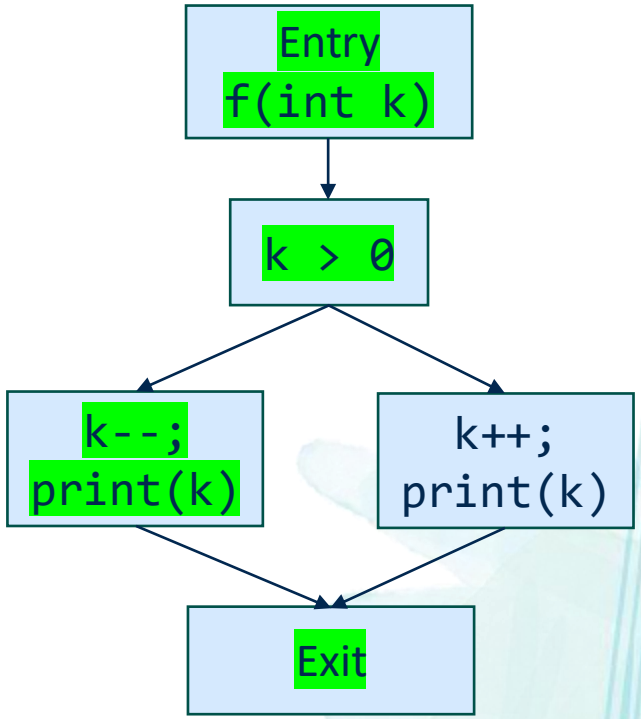
BB-level coverage

- Fuzzers record coverage at the BB level
- For every input, the fuzzer records which BBs are covered
- If new BBs are covered, it considers the input “interesting”
- E.g. `f(10)`



BB-level coverage

- Fuzzers record coverage at the BB level
- For every input, the fuzzer records which BBs are covered
- If new BBs are covered, it considers the input “interesting”
- E.g. `f(10)`

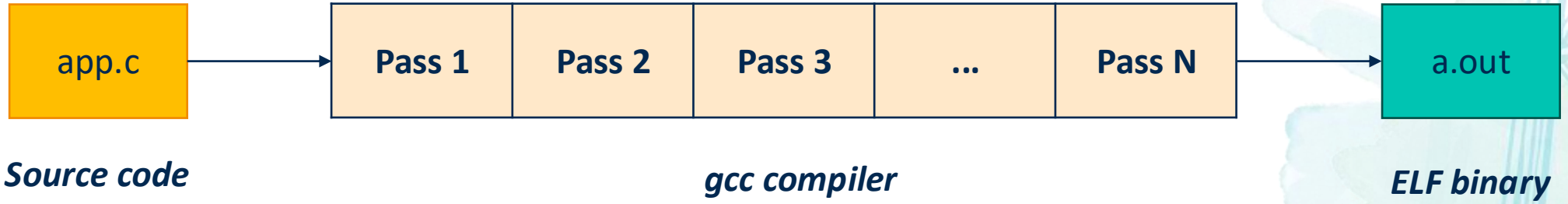


Compilers

Many automated software engineering techniques, including fuzzing, leverage the compiler

Compilers

- Compiler consists of “passes” that analyze and transform the code
- Each pass gradually lowers the source code to assembly
  - Produces many program representations, including CFG



Coverage instrumentation

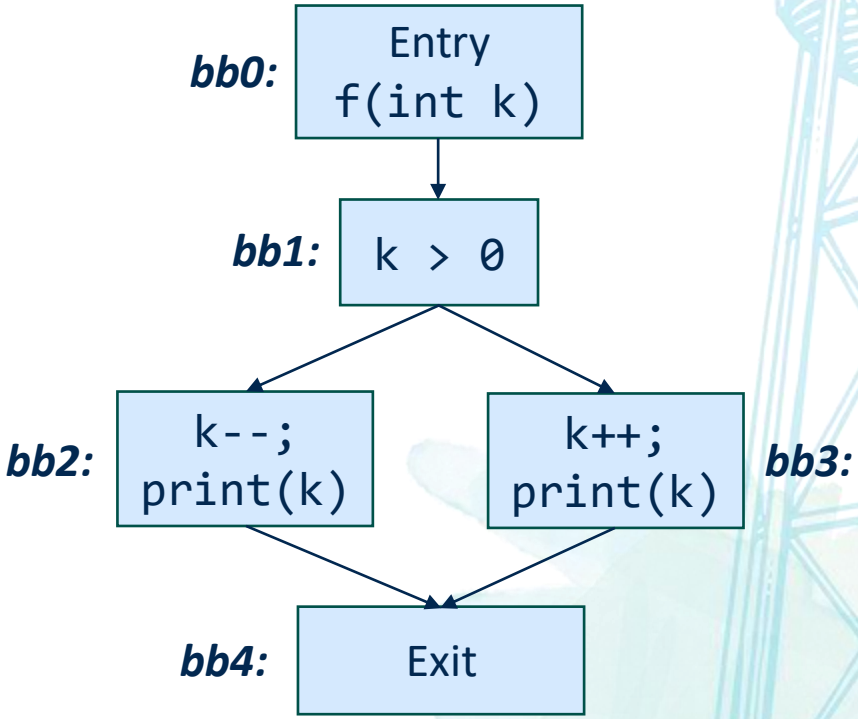
- `afl-gcc` is a wrapper around `gcc`
- Inserts additional passes into the compiler pipeline

Coverage instrumentation

- AFL compiler instruments the CFG edge-transition to record coverage
- Adds a coverage map

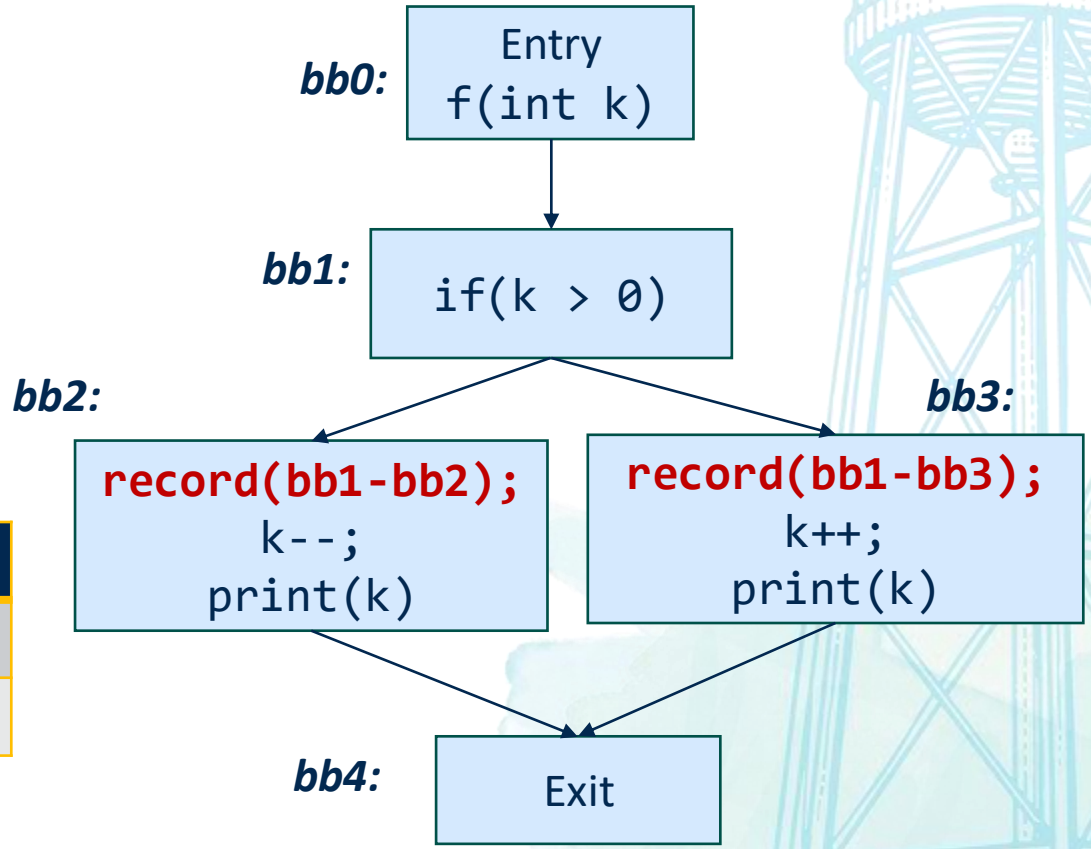
Edge	Coverage
bb1-bb2	
bb1-bb3	

Coverage map



Coverage instrumentation

- AFL compiler instruments the CFG edge-transition to record coverage
- Adds a coverage map



Edge	Coverage
bb1-bb2	
bb1-bb3	

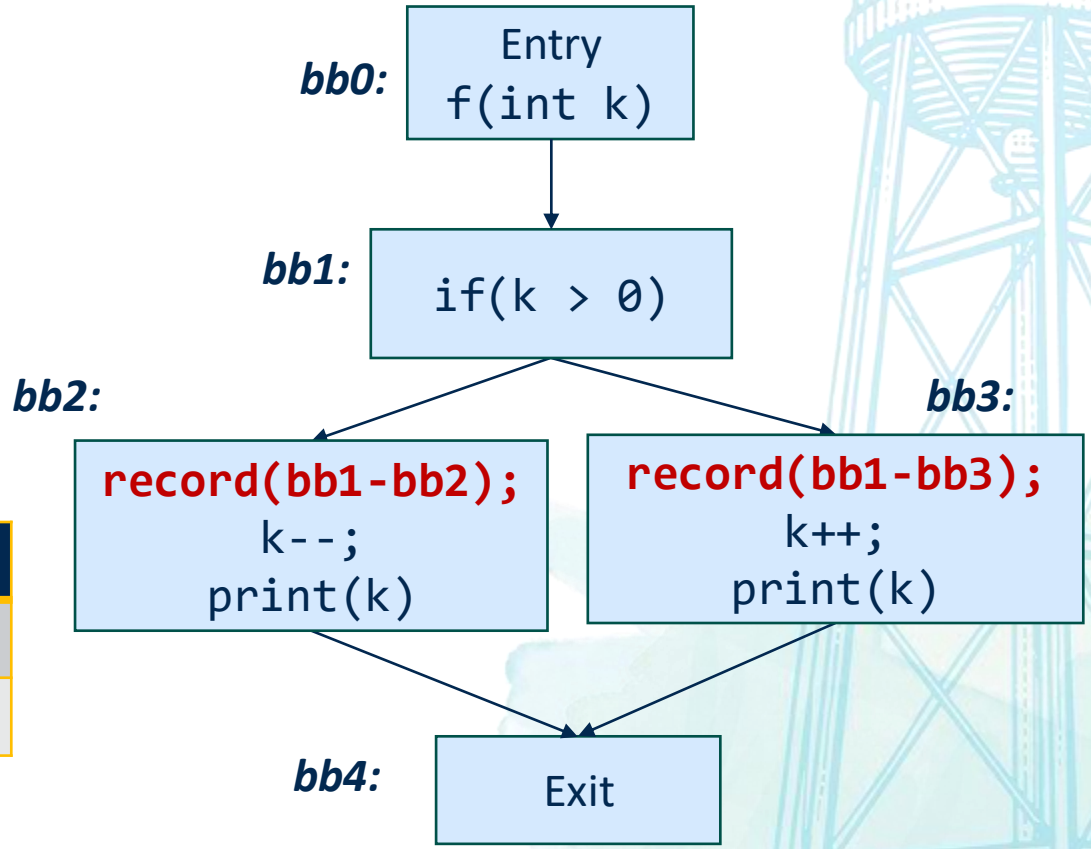
Coverage map

Coverage recording during execution

- Instrumentation records the coverage in the coverage map
- Fuzzer reads the coverage map to determine which edges covered
- E.g. `f(10)`

Edge	Coverage
bb1-bb2	
bb1-bb3	

Coverage map

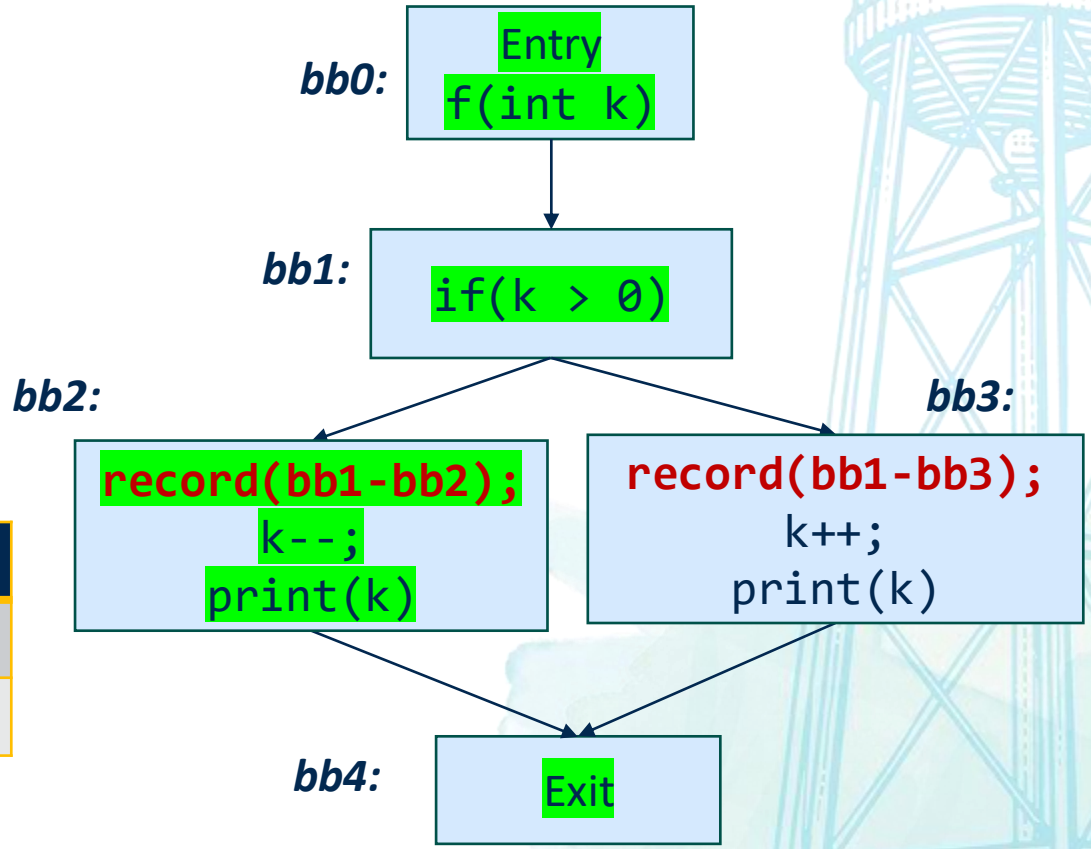


Coverage recording during execution

- Instrumentation records the coverage in the coverage map
- Fuzzer reads the coverage map to determine which edges covered
- E.g. `f(10)`

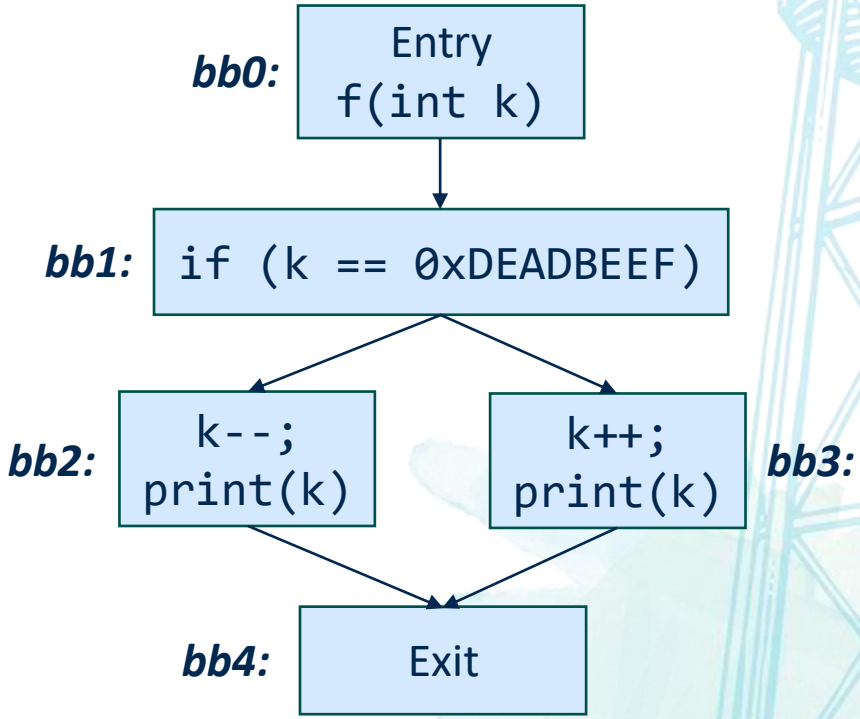
Edge	Coverage
bb1-bb2	1
bb1-bb3	

Coverage map



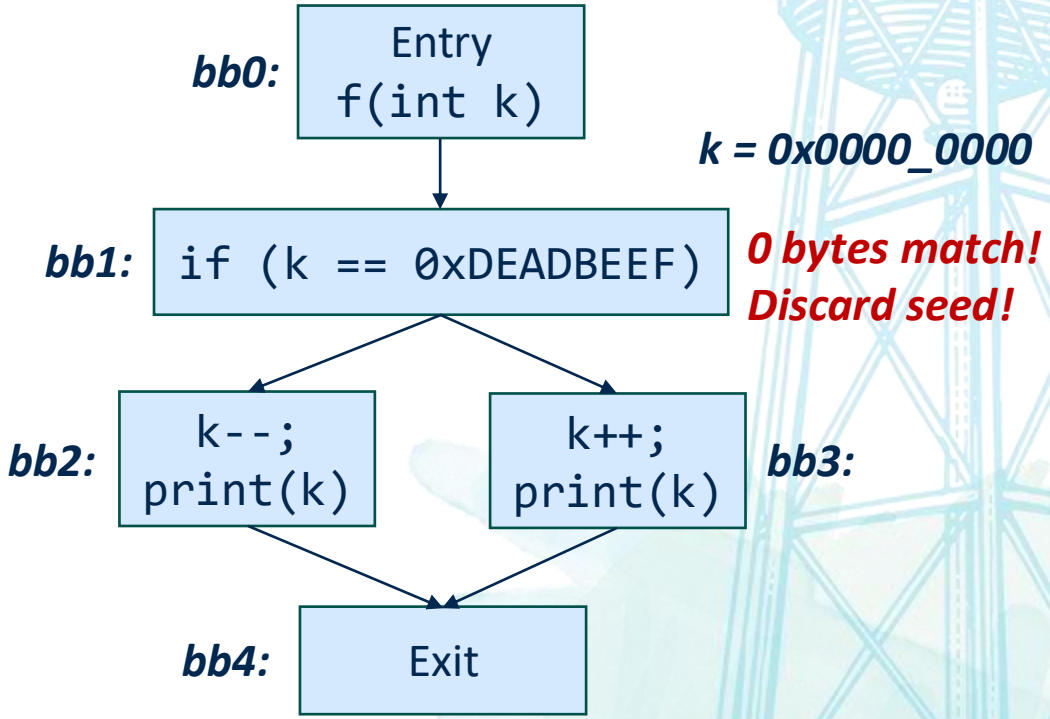
Branch coverage limitation

- E.g., branch condition `if (input == 0xDEADBEEF)`
- Branch coverage sees "not taken" for *every* input except the magic value
- Hard to distinguish 0xDEADBEE0 (close!) from 0x00000000 (far)
- Random mutation is very unlikely to hit the exact value



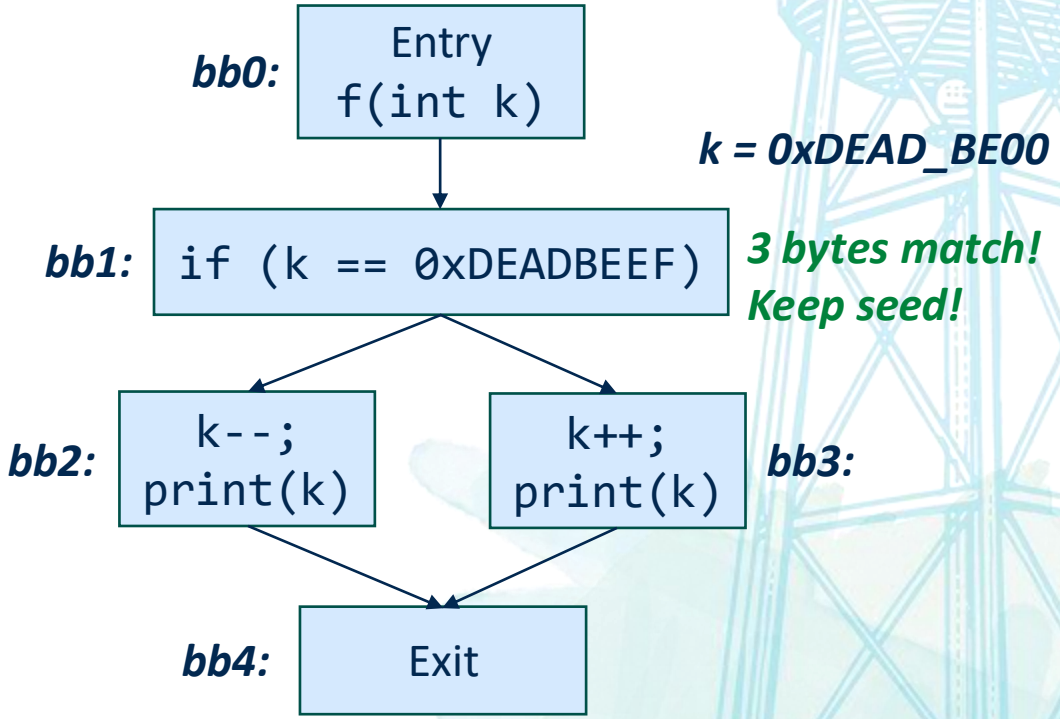
Value profile coverage

- Track the *values* being compared, not just the branch outcome
- For `if (x == 0xDEADBEEF)`, count how many bytes of x match the target
- E.g., `f(0x0000_0000)`
- Used by libfuzzer



Value profile coverage

- Track the *values* being compared, not just the branch outcome
- For `if (x == 0xDEADBEEF)`, count how many bytes of `x` match the target
- E.g., `f(0xDEAD_BE00)`
- Used by libfuzzer
- Value profile gradually "walks" toward the target value



Path coverage

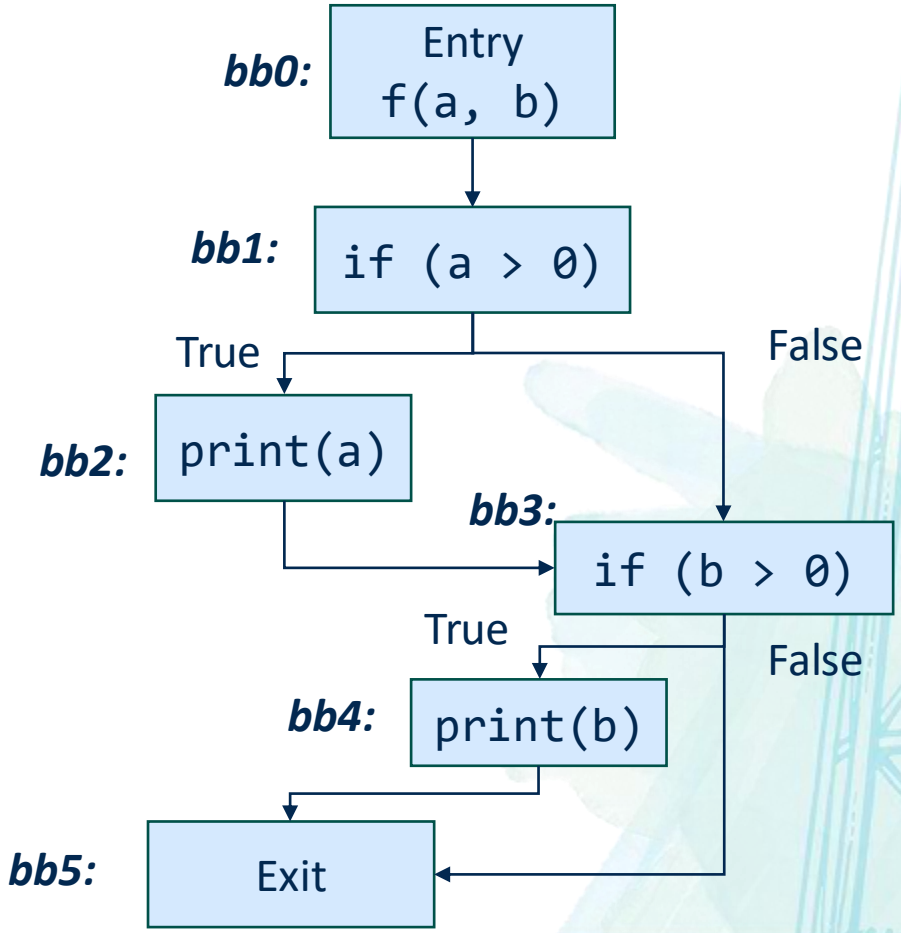
- Branch coverage: did we take this branch (yes/no)?
- Path coverage: track the *exact sequence* of branches taken
- Same branches, different order = different path

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```

Path coverage

- Branch coverage: did we take this branch (yes/no)?
- Path coverage: track the *exact sequence* of branches taken
- Same branches, different order = different path
- Bugs might depend on the exact path

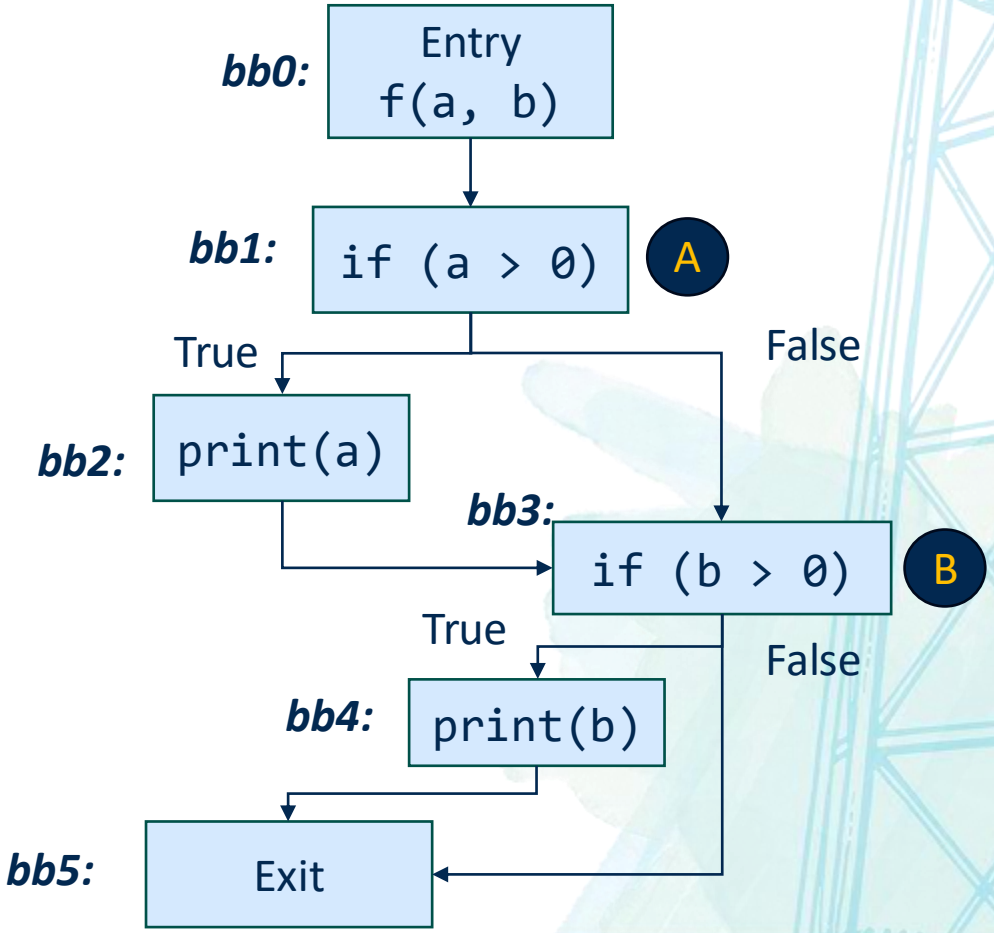
```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



Path coverage

- Has 4 paths
  - A not taken, B not taken
  - A not taken, B taken
  - A taken, B not taken
  - A taken, B taken

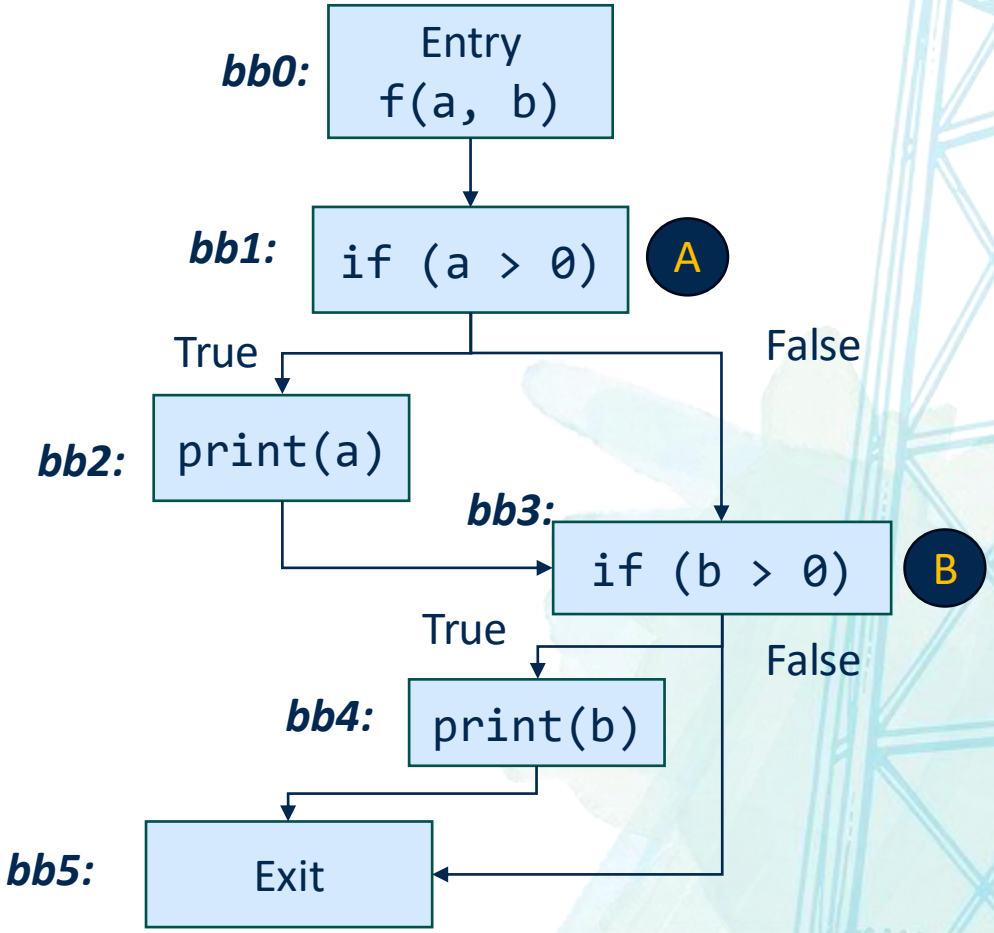
```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



Path coverage vs edge coverage

- Edge coverage only records whether an edge was traversed
  - Ignores execution history

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



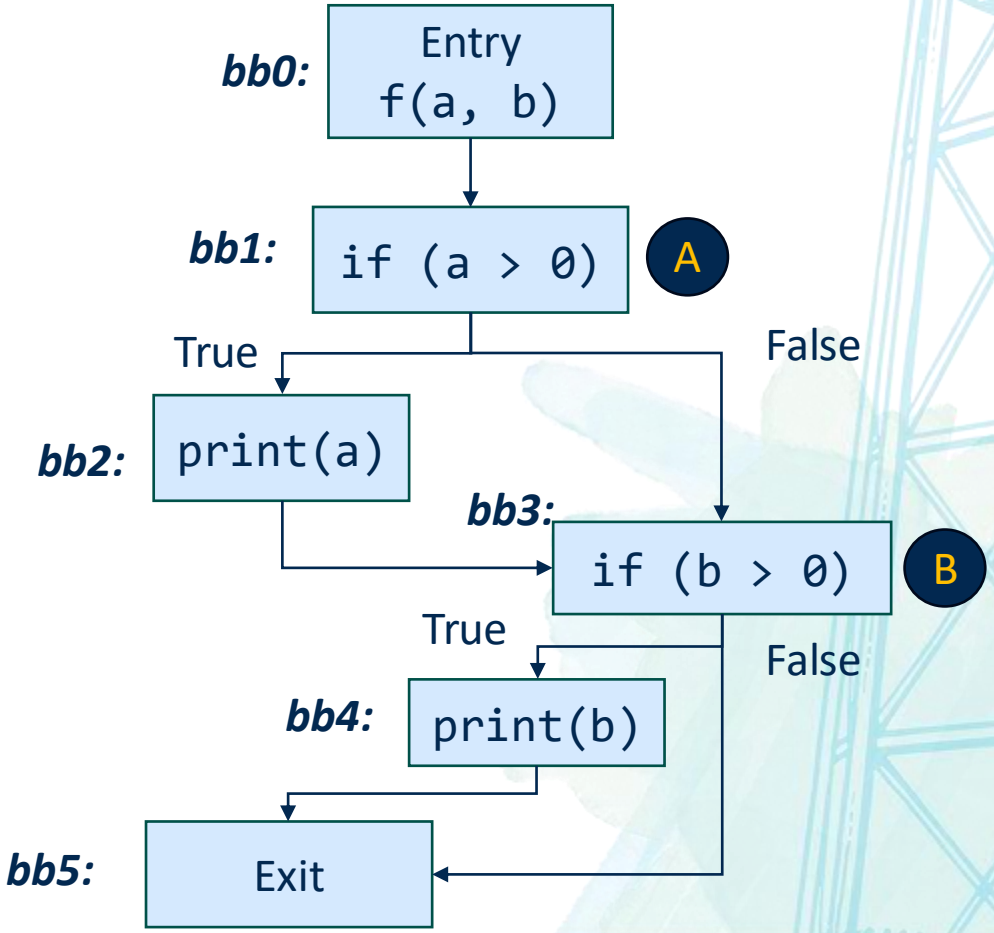
Coverage map

Edge	Coverage
bb1-bb2	
bb1-bb3	
bb3-bb4	
bb3-bb5	

Path coverage vs edge coverage

- Edge coverage only records whether an edge was traversed
  - Ignores execution history
- Eg. a = 10, b = -1

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



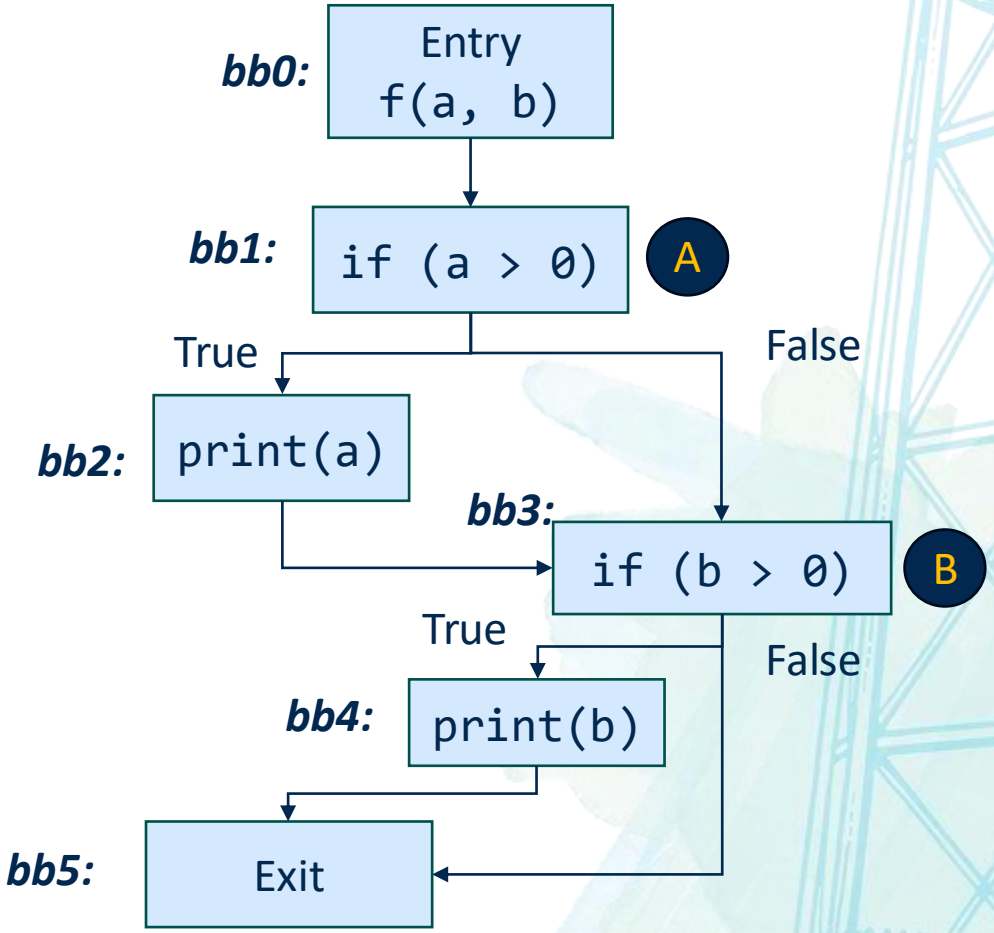
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	0
bb3-bb4	0
bb3-bb5	1

Path coverage vs edge coverage

- Edge coverage only records whether an edge was traversed
  - Ignores execution history
- Eg. a = -1, b = 10

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



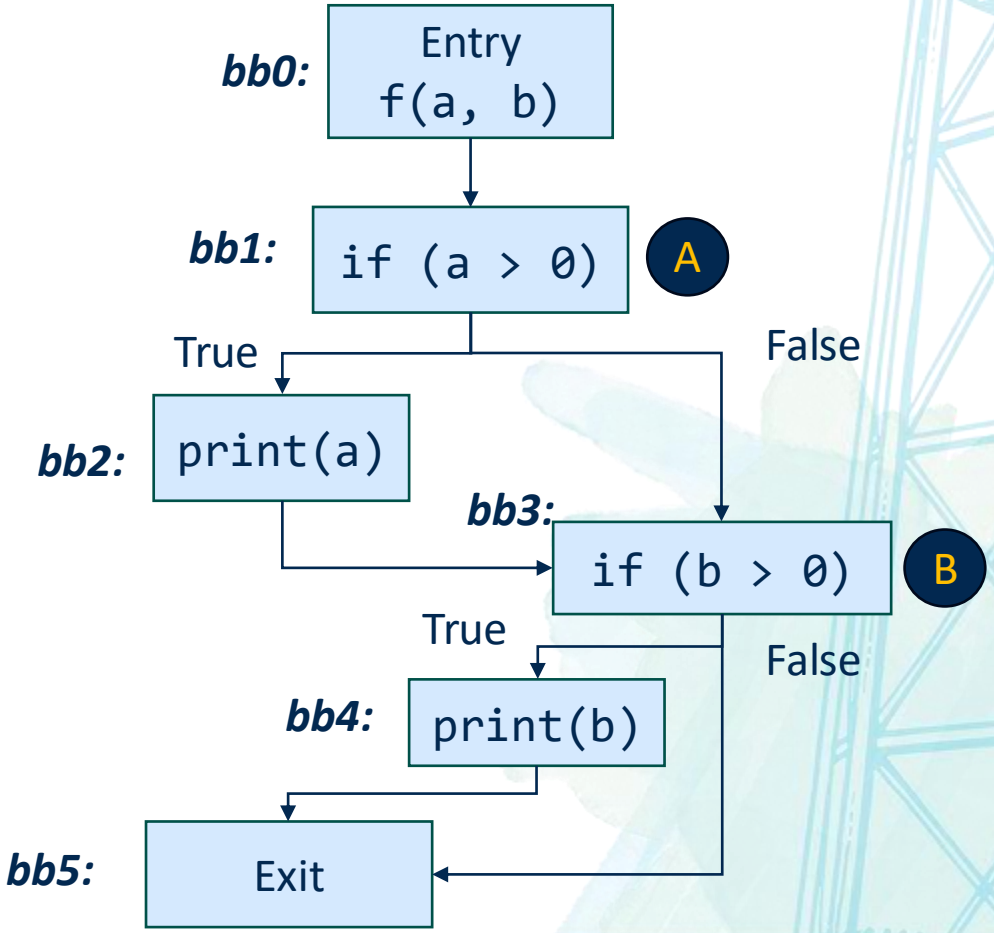
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage vs edge coverage

- Two inputs result in "full" coverage
- But explores only 2 of 4 paths

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



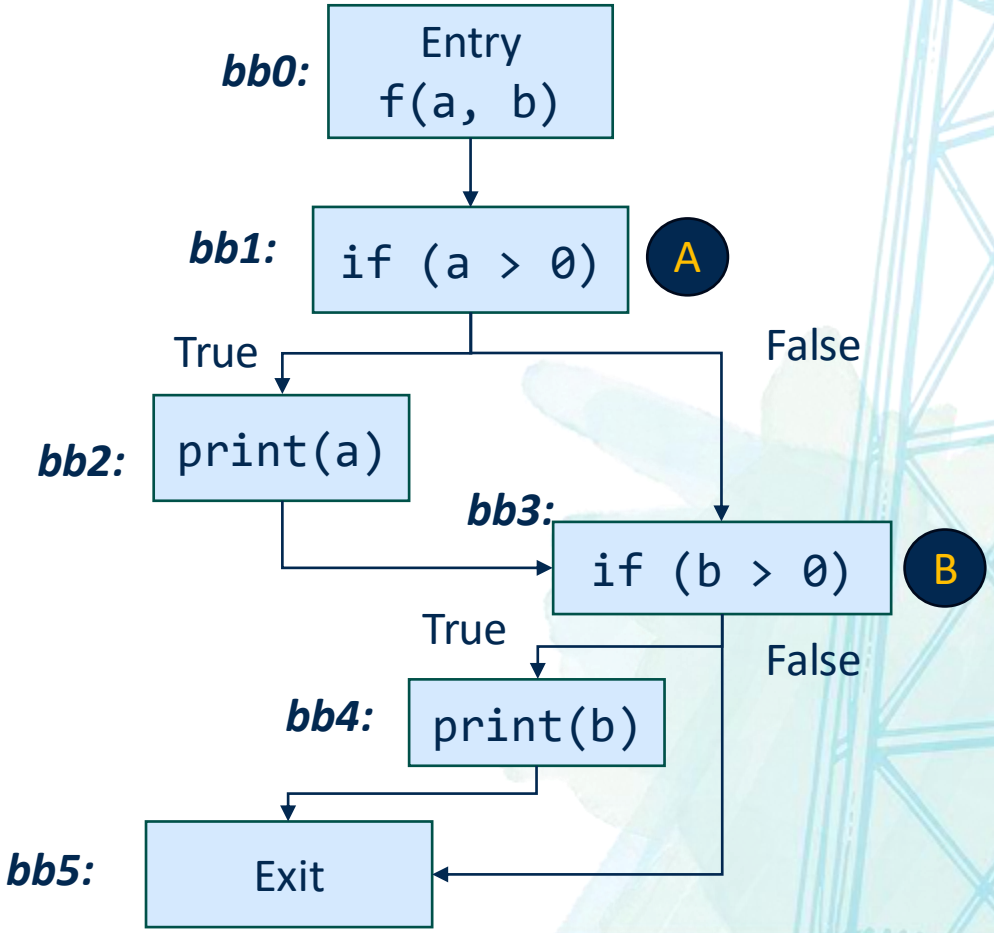
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage vs edge coverage

Path	Explored?
A not taken, B not taken	No
A not taken, B taken	Yes
A taken, B not taken	Yes
A taken, B taken	No

```
if (a > 0) { print("a"); }    // branch A
if (b > 0) { print("b"); }    // branch B
```



Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage

- A path-coverage based fuzzer records path-coverage instead of just edge-coverage
- Stores significantly more information per execution
- Leads to path explosion
  - With N branches, up to 2^N possible paths
  - Loops make matters worse – every iteration is a new path!
- Path coverage is the most precise but impractical to fully achieve
  - Industrial fuzzers like AFL only use edge-coverages

Coverage granularity spectrum

- Branch/edge coverage: which branches were taken (AFL)
- Value profile: how close comparisons are to being satisfied (libFuzzer)
- Path coverage: exact sequence of branches (most precise, but exponential blowup)
- Fine-grained coverage has a higher performance overhead
- Tradeoffs depend on your application!

Fuzz testing - sanitizers

Tapti Palit

Recall: memory safety bugs

- Fuzzers execute the program with random inputs and look for crashes
- But memory safety bugs may or may not crash the program
- **Undefined Behavior (UB)**

Sanitizers: oracles for memory safety bugs

- Adds additional compiler instrumentation to record object metadata
- Turns silent UBs to crashes
- Popular sanitizers
  - Address sanitizer (ASAN) – can detect buffer overflow
  - Leak sanitizer (LSAN) – can detect memory leak
  - Thread sanitizer (TSAN) – can detect data races
- ***Significantly*** slows down the application; but can detect memory safety bugs!!

Address sanitizer: high level idea

- Goal – detect buffer overflow
- During program execution, what information is needed to determine if `function()` causes a buffer overflow?

```
void function(char* p) {
    p[0] = ‘h’;
    p[1] = ‘e’;
    p[2] = ‘l’;
    p[3] = ‘l’;
    p[4] = ‘o’;
    p[5] = ‘\n’;
}

int main(void) {
    char* s = malloc(5);
    function(s);
}
```

Address sanitizer: high level idea

- Address sanitizer is a compiler tool
- When compiling the program, it adds code to
  - Record the size of heap buffers
  - Check if an out-of-bounds access occurs

```
void function(char* p) {
    p[0] = 'h';
    p[1] = 'e';
    p[2] = 'l';
    p[3] = 'l';
    p[4] = 'o';
    p[5] = '\n';
}

int main(void) {
    char* s = malloc(5);
    function(s);
}
```

Check if each offset is within the heap object size

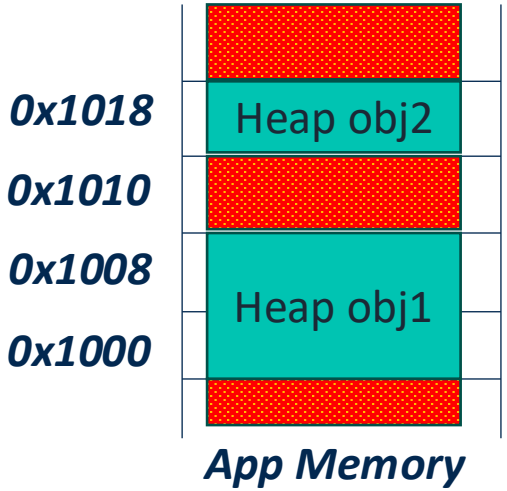
Must record the size of the allocated heap object

ASan

- ASan is a sanitizer used with popular fuzzers (AFL, libfuzzer)
  - Goal: low overhead
- Insert “red/poison zones” around memory
- Updates to red zones indicate buffer overflow

```
void function(char* p) {
    while(*p != ‘\0’) *p++ = ‘a’;
}

int main(void) {
    char* s = malloc(16); // heap object 1
    char* p = malloc(8); // heap object 2
    function(s);
}
```

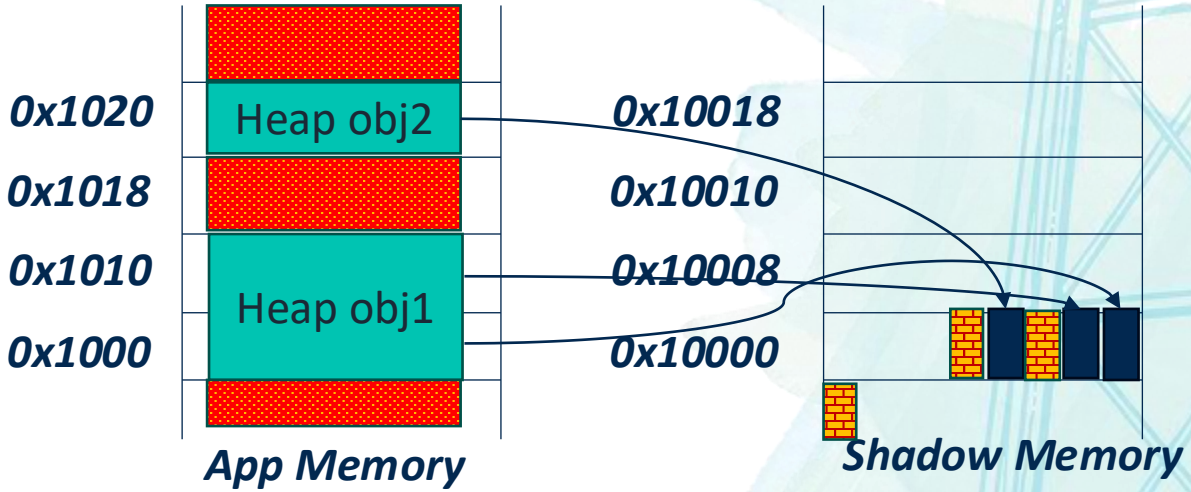


Shadow memory

- Maintains shadow memory to record metadata about red and accessible zones
- Each ***byte*** in shadow memory represents a corresponding ***8 bytes*** in app
  - 0x00 for valid
  - 0xFF for poisoned range
  - 0x01 – 0x07 – partially poisoned

```
void function(char* p) {
     while(*p != '\0') *p++ = 'a';
}


int main(void) {
    char* s = malloc(16); // heap object 1
    char* p = malloc(8); // heap object 2
    function(s);
}
```

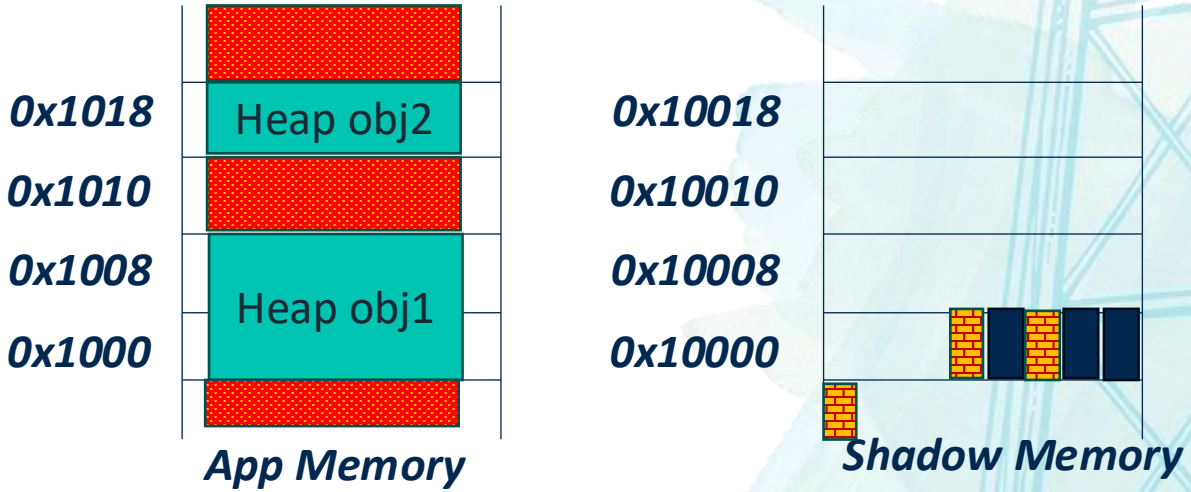


Compiler checks

- Compiler adds instructions before each memory access to first check the shadow memory byte

```
void function(char* p) {
     while(*p != '\0') *p++ = 'a';
}


int main(void) {
    char* s = malloc(16); // heap object 1
    char* p = malloc(8); // heap object 2
    function(s);
}
```

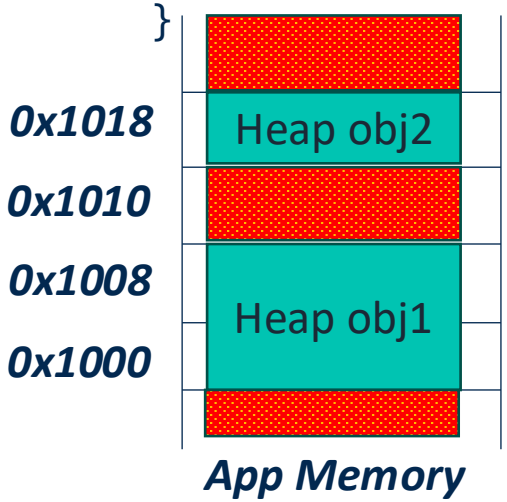


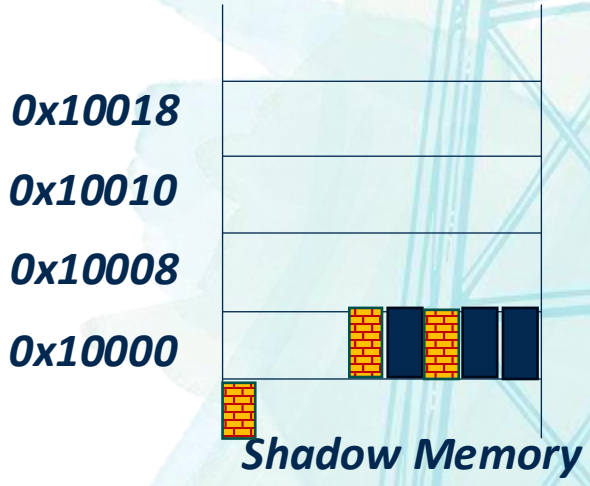
Compiler checks pseudocode

- Compiler adds instructions before each memory access to first check the shadow memory byte
- Pseudocode
  - shdw_poisoned(void*) computes the address of the shadow byte
  - Checks if it is fully/partially poisoned

```
void function(char* p) {
     while(*p != '\0') {
        if (shdw_poisoned(p)) { assert(false); }
        *p++ = 'a';
     }
}

int main(void) {
    char* s = malloc(16); // heap object 1
    char* p = malloc(8); // heap object 2
    function(s);
}
```





Why shadow memory?

- Shadow memory stores metadata about the 8-byte memory range
- Fast computation of shadow byte address
  - `shadow = (address >> 3) + SHADOW_OFFSET`
- Fast checks of shadow byte value
  - `shadow_value = *shadow`

Fuzzer success factors

- Fuzzer throughput (exec/s)
- Percentage of valid inputs generated by the mutator
- Type of coverage recorded
- Sanitizer vs. no sanitizer
- Ideally, want a high percentage of valid inputs + fine-grained code and data coverage + sanitizers
  - Unfortunately reduces throughput
- Tradeoff: finding the right balance for your use case!

Summary

- Fuzzing is an effective testing mechanism
- Unlike other testing approaches it can ***find new inputs*** that cause program misbehavior
- Mutation and coverage are important for generating effective fuzzers
- Sanitizers provide additional information needed to find *silent UB* bugs